From Entities to Answers: Graph-Based Retrieval for Intelligent News Analytics

Michael Shell, *Member, IEEE,* John Doe, *Fellow, OSA,* and Jane Doe, *Life Fellow, IEEE*‡

June 12, 2025

***Abstract*—The abstract goes here.**

***Index Terms*—IEEE, IEEEtran, journal, LaTeX, paper, template.**

I. Introduction

In today's fast-paced, information-driven world, extracting meaningful insights from the overwhelming volume of online news remains a significant challenge. Traditional approaches often struggle to interpret the complex relationships, evolving timelines, and contextual nuances embedded within news articles. *News Analytics as a Service (NAaaS)* is proposed as a one-stop solution to address these challenges by integrating advanced Natural Language Processing (NLP), Knowledge Graphs, and AI-driven retrieval techniques to deliver a rich, temporal and geospatial understanding of news content.

At its core, NAaaS provides users with rapid access to relevant news articles within a defined time frame and related to a specific event or topic. It then plots these news instances on an interactive **Web-based Geographic Information System (GIS)**, enabling users to visualize their geographical distribution across Pakistan, down to the granularity of districts. This visual mapping significantly enhances comprehension of the spatial relevance of news and supports location-aware information exploration.

To process and scale this vast amount of data, NAaaS is built using distributed computing principles and big data tools. Its backend architecture operates over a cluster of computers that allows for real-time stream processing, efficient query execution, and robust handling of high-volume news data. This technical foundation ensures the system remains responsive and scalable.

A key innovation in this project is the integration of an **AI-driven module called NewsNet GraphRAG**. This module is designed to overcome the limitations of traditional text retrieval and summarization by incorporating *Retrieval-Augmented Generation (RAG)* combined with Knowledge Graphs [1], [2]. NewsNet GraphRAG offers a deeper contextual understanding of news articles, identifying relationships between entities, events, times, and places. This hybrid model not only improves temporal analysis but also supports the construction of enriched knowledge graphs that reflect the interconnections found in real-world narratives. It facilitates a more comprehensive semantic retrieval experience by enabling context-aware reasoning over extracted entities, relations, and document timelines [3], [4].

Understanding information is inherently influenced by time, as new topics emerge and existing ones evolve. Concurrently, users' information-seeking behavior also shifts, affecting their expectations from Information Retrieval (IR) systems. Temporal understanding in news documents is represented through both **explicit** and **implicit** temporal expressions [5]. Explicit expressions (e.g., "October 8, 2005") are clearly embedded in the content and metadata, while implicit expressions (e.g., "D-day", "last week", "Friday") require contextual interpretation. NAaaS identifies and differentiates between these forms to enhance the accuracy of focus time extraction, enabling a more precise timeline reconstruction for news events.

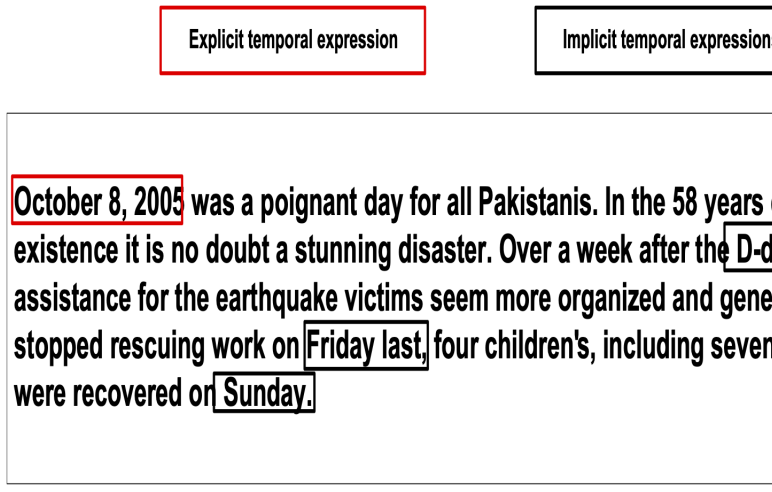


Fig. 1: Explicit and Implicit temporal expressions in the text of news document related to earthquake event occurred in Pakistan in 2005.

*M. Shell was with the Department of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA, 30332 USA e-mail: (see http://www.michaelshell.org/contact.html).

†J. Doe and J. Doe are with Anonymous University.

‡Manuscript received April 19, 2005; revised August 26, 2015.

News documents consist of a *focus time* (time/date the news is referring to), *focus location* (location to which the news is referring to), and *creation time* (time/date the news was created or uploaded). However, it has been observed that the focus time and focus location often do not relate directly to the document's creation time or the area mentioned in the headline. Consequently, it is usually left to the reader to interpret the temporal and spatial specificity of the news article. NAaaS addresses this gap by programmatically extracting and specifying the focus time and location, and then plotting the information on a map of Pakistan [6].

We argue that the focus time of a document holds significant importance in fully meeting a user's temporal intentions. Efficient techniques are required to leverage the content of temporal documents effectively, ensuring the retrieval of the most accurate and relevant documents in response to user queries. Current information retrieval (IR) systems lack the capability to achieve this level of precision.

Named Entity Recognition (NER) holds paramount significance in the domain of news analysis and document clustering. Entity-aware clustering approaches have recently been explored to embed semantic relationships among documents [6]. In NAaaS, NER plays a foundational role in extracting meaningful entities from articles, which are then used in the construction of knowledge graphs.

Sentiment analysis also plays a crucial role in NAaaS by providing valuable insights into the sentiment expressed in news articles. By analyzing sentiment timelines, the system supports actionable understanding of public opinion. These trends can be used to correlate emotional tone with evolving events and political contexts, thus enriching the knowledge graphs with sentiment-based edges and weights [7].

II. Literature Review

Deliverable 1

We have analyzed the methodologies and tools presented to gain a firm understanding of the *journalism, natural Language processing, knowledge graphs and distributed processing* . The predominant parameters explored in the reviewed literature include temporal and spatial taggers, expression extractors, expression annotators, part-of-speech (PoS) taggers, and Named Entity Recognition (NER). Time-related discussions hold significant importance across various domains within the social sciences, as well as in official policy announcements. While Natural Language Processing (NLP) techniques have gained traction in addressing social science research questions, there has been limited effort to quantitatively measure temporal expressions[8].

We can identify three fundamental dimensions for understanding and analyzing text, often termed as the *three Ts of text*[9]. The first dimension, time dimensionality, focuses on the temporal aspects within text, including creation time, historical context, and events described. Understanding these temporal markers is crucial for historical analysis and tracking changes over time. The second dimension, topic, pertains to the subject matter or theme of the text, enabling categorization and organization for better analysis. Lastly, the tone dimension encompasses the emotional or attitudinal qualities expressed in the text, crucial for understanding the author's perspective and evaluating sentiment. These dimensions collectively provide insights into the structural, thematic, and emotional aspects of textual data, facilitating a comprehensive understanding of the text.

Authors in [10] outline a vision for analyzing semantic trajectory traces and generating synthetic semantic trajectory data (SSTs) utilizing generative language models. Drawing on the advancements in deep learning witnessed in fields like natural language processing (NLP) and computer vision, the aim to develop intelligent models capable of studying semantic trajectories across diverse contexts. These models will predict future trends, enhance machine understanding of movements involving animals, humans, goods, etc., improve human-computer interactions, and facilitate a wide range of applications spanning urban planning, personalized recommendation engines, and business strategy.

In [11], authors capitalize on the intrinsic nature of time, organizing all temporally-scoped sentences along a one-dimensional time axis and proposing the creation of a graph structure based on the relative positioning of events along this axis. Inspired by this graph-based perspective, authors introduce RemeMo (Relative Time Modeling), a model that explicitly establishes connections between all temporally-scoped facts by modeling the temporal relations between any two sentences. Experimental findings demonstrate that RemeMo outperforms the baseline T5 across multiple temporal question answering datasets and various experimental settings. Furthermore, detailed analysis reveals RemeMo's particular proficiency in modeling intricate, long-range temporal dependencies.

Understanding and effectively addressing cybercrime necessitates accurate measurement and mapping to inform crime reduction strategies, identify strategy gaps, and guide ongoing research efforts. This thesis endeavors to develop and assess methodologies for detecting, preventing, and com-

prehending spatio-temporal patterns of cybercrime, with a specific focus on malicious download events and IoT botnet attacks. By harnessing natural language processing (NLP) techniques, such as word embeddings, and deep learning methodologies for time series analysis, such as long short-term memory (LSTM) networks, in conjunction with principles from technical analysis in financial trading, the aim is to offer insights into cybercrime detection and prevention. Defining and comprehending cyberspace poses a formidable challenge in the study of cybercrime. While classification frameworks for cybercrime are well-established and sustainable in the long run, establishing a unified definition and taxonomy for cyberspace itself proves difficult due to its volatile nature and the myriad of potential frameworks. To tackle this challenge, a bottom-up design approach is adopted, wherein cyberspace is defined based on the type of cybercrime under investigation, data accessibility, research objectives, and the researcher's ingenuity. The approach [12]facilitates a more practical and context-specific understanding of cyberspace within the realm of cybercrime.

Temporal commonsense reasoning involves understanding the typical temporal context surrounding phrases, actions, and events, and leveraging this understanding to tackle problems requiring such knowledge. This skill is crucial in temporal natural language processing tasks, with potential applications including timeline summarization, temporal question answering, and temporal natural language inference. Recent studies on large language models indicate that while they excel at generating syntactically correct sentences and completing classification tasks, they often resort to shortcuts in their reasoning and fall into simple linguistic traps. This article[13] offers an overview of research in the field of temporal commonsense reasoning, particularly focusing on enhancing language model performance through various augmentations and evaluating them across an expanding array of datasets. However, despite these enhancements, augmented models still struggle to match human performance in reasoning tasks involving temporal commonsense properties, such as typical occurrence times, orderings, or durations of events.

Numerous facts undergo changes over time, leading to the emergence of Time-sensitive Question Answering (TSQA) tasks, which assess models' proficiency in addressing temporal aspects as identified in[14]. While existing methods excel in handling straightforward questions concerning specific moments, they often struggle with more complex queries that require understanding temporal relationships and numeric comparisons. To address this challenge, Temporal-aware Multitask Learning (TML) with three auxiliary tasks is introduced. Firstly, a temporal-aware sequence labeling task is proposed to aid in distinguishing temporal expressions by identifying temporal types of tokens within documents. Secondly, a temporal-aware masked language modeling task to capture temporal relations between events based on contextual information is deployed. Lastly, temporal-aware order learning is introduced to enhance the model's capacity for numeric comparison. Through comprehensive experiments conducted on the TimeQA benchmark, authors assess the effectiveness of our proposed methodology in handling temporal question answering. Results demonstrate that TML outperforms baseline approaches by a substantial relative improvement of 10% across the dataset's two splits.

Community Question Answering (CQA) platforms facilitate knowledge sharing by enabling users to post questions and receive answers from other users. Identifying high-quality answers amidst a pool of responses is a challenging task within Natural Language Processing (NLP). In response, authors in [15] introduce a deep learning-based spatial-temporal Bidirectional Long Short-Term Memory (Bi-LSTM) algorithm. While existing approaches predominantly focus on computing semantic similarity between questions and answers using user votes, our hybrid approach incorporates both forward and backward LSTM networks to assess question-answer and answer-answer similarity. The forward LSTM captures spatial relationships to estimate relevance, while the backward LSTM learns temporal features to predict the best quality answer. Additionally, spatial Bi-LSTM models past and future dependencies to enhance context comprehension and answer selection effectiveness. To extract meaningful information from noisy text data, standard preprocessing steps such as tokenization, parsing, lemmatization, stop words removal, part-of-speech tagging, and entity extraction are applied. Furthermore, Word embeddings-based Paragraph-to-Vector (par2vec) includes additional input nodes to represent paragraph information as vectors for improved context comprehension. Empirical analysis conducted on the SemEval CQA dataset demonstrates that our proposed model outperforms existing state-of-the-art answer selection approaches.

A. Text processing and Journalism

The inverted pyramid news paradigm is a traditional approach to structuring news articles, commonly employed in journalism. In this paradigm, the most essential information is presented at the

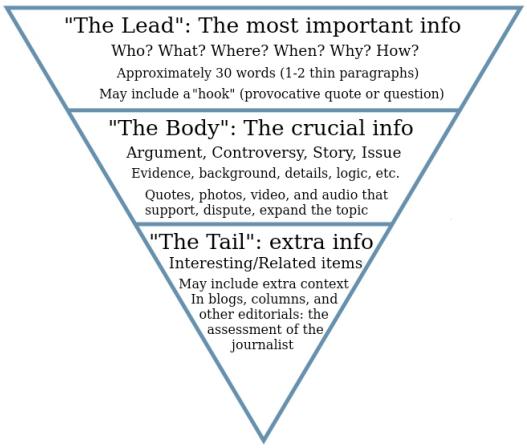


Fig. 2: Inverted pyramid news paradigm[16].

beginning of the article, followed by additional details arranged in descending order of importance as shown in in Fig: 2. While the inverted pyramid remains a widely used and respected paradigm in journalism, especially for breaking news stories, there are other narrative structures and storytelling formats that have gained prominence in modern journalism. One such approach is the "*nut graph* or *summary lead* technique[16]. The nut graph technique involves providing a concise summary or analysis of the main points of the story within the first few paragraphs, often after an attention-grabbing lead. This approach aims to engage readers quickly by providing context and relevance upfront before delving into the details of the story. The nut graph is similar to the inverted pyramid in that it prioritizes key information early on but differs in its focus on providing context and analysis alongside the core facts. Additionally, narrative journalism has become increasingly popular in recent years. This approach prioritizes storytelling techniques more commonly associated with literature, such as vivid descriptions, character development, and narrative arcs. Narrative journalism seeks to immerse readers in the story by weaving together facts, quotes, and anecdotes into a compelling narrative structure. While not a replacement for the inverted pyramid, narrative journalism offers an alternative approach to traditional news reporting, particularly for feature stories and long-form articles.

Big Data tools like Apache Kafka[17] and Apache Spark[18] are used together to process huge amount of streaming or batch data.

B. Distributed Stream Processing

Apache Kafka is a distributed streaming platform that is used to build real-time data pipelines and streaming applications. It is designed to handle high-throughput, fault-tolerant, and scalable messaging. Kafka [17] enables the publishing and subscribing to streams of records, similar to a message queue or enterprise messaging system. It stores streams of records in a fault-tolerant way and processes streams of records as they occur.

Key features of Kafka include:

- **Durability:** Data is written to disk and replicated within the cluster to ensure durability.
- **Scalability:** Kafka scales easily by adding more brokers to the cluster.
- **High Throughput:** Capable of handling millions of messages per second.
- **Fault Tolerance:** Data replication ensures no single point of failure.
- **Real-time Stream Processing:** Allows real-time data ingestion and processing.

Apache Spark [18]is an open-source unified analytics engine for large-scale data processing. It provides an interface for programming entire clusters with implicit data parallelism and fault tolerance. Spark offers high-level APIs in Java, Scala, Python, and R, and an optimized engine that supports general execution graphs. It also includes libraries for SQL, streaming, machine learning, and graph processing.

Key features of Spark include:

- **Speed:** In-memory computation boosts processing speed.
- **Ease of Use:** Simple APIs in multiple programming languages.
- **Generality:** Can handle a wide range of tasks, including batch processing, stream processing, and interactive queries.

When Kafka is used in conjunction with Spark, it provides a powerful combination for real-time stream processing. Kafka serves as the data ingestion layer, handling the real-time data feeds, while Spark processes and analyzes this data in real-time. This integration allows for the creation of complex, fault-tolerant, and scalable real-time data processing pipelines. Some real use cases are:

The discussion on temporal expression identification underscores the importance of understanding natural language texts, particularly in large-scale applications and production environments. While highly effective systems like HeidelTime exist, their limited runtime performance poses challenges for widespread adoption. However, our introduction of the TEI2GO models addresses this issue by matching HeidelTime's effectiveness while significantly improving runtime efficiency. These models support six languages and achieve state-of-the-art results in four of them, making them a valuable resource for temporal expression identification tasks.

To train the TEI2GO models, authors employed a combination of manually annotated reference corpora and developed the ”Professor HeidelTime” corpus, a comprehensive weakly labeled dataset of news texts annotated with HeidelTime. This corpus, comprising over 138,000 documents in six languages and containing 1,050,921 temporal expressions, stands as the largest open-source annotated dataset for temporal expression identification to date. By detailing the methodology behind the development of these models, our aim is to inspire the research community to further explore, refine, and expand the range of models available for additional languages and domains. Authors have made annotations, and models openly accessible for community exploration and use. These models are conveniently hosted on HuggingFace for seamless integration and application, providing researchers and practitioners with the tools they need to effectively identify temporal expressions in natural language texts across various domains and languages. This discussion sets the stage for the subsequent section, which explores the utilization of Apache Spark for processing natural language data, leveraging the advancements made in temporal expression identification with TEI2GO models[19].

Kafka and Spark are often integrated together due to their complementary features and capabilities. Both systems are designed to operate in distributed environments, allowing them to scale horizontally across multiple nodes efficiently while maintaining fault tolerance. Kafka serves as a distributed messaging system, facilitating real-time data streaming by acting as a buffer between data producers and consumers[20]. This decouples data producers from consumers, enabling each component to operate independently and ensuring that data ingestion and processing tasks can proceed at their own pace without affecting each other as shown in Fig: 3. One of Kafka’s key features

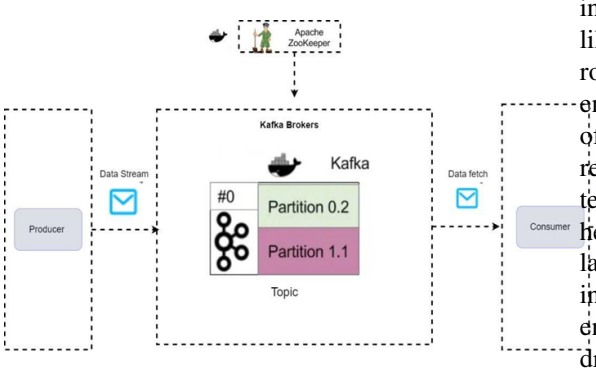


Fig. 3: Kafka Spark Integeration Architecture[21]

is its ability to provide built-in mechanisms for data durability and replication, ensuring that data is not lost in case of failures. This feature is essential for building fault-tolerant data processing pipelines, which Spark can leverage to ensure data integrity. Spark, on the other hand, seamlessly integrates with Kafka through its Spark Streaming component[22], allowing Spark applications to consume data from Kafka topics directly. This integration simplifies the development of real-time data processing pipelines by enabling developers to use familiar Spark APIs[21].

Moreover, many organizations use both Kafka and Spark as part of their analytics stack, creating a unified platform that combines real-time data streaming, batch processing, and advanced analytics capabilities. This unified platform enables a wide range of use cases, from data ingestion to machine learning model training. Overall, the integration of Kafka with Spark provides organizations with a powerful and flexible infrastructure for building scalable, real-time data processing pipelines, making it a popular choice for handling streaming data and deriving insights from it[23].

C. Knowledge Graphs and Named Entity Recognition

NER plays a pivotal role in this process by identifying and classifying entities of interest, such as Persons, Organizations, and Locations, within the text. This capability is crucial for understanding the semantics of documents, guiding their interpretation, and facilitating downstream tasks such as information retrieval and extraction. In the context of News, the importance of NER is particularly pronounced, as it enables the retrieval of documents and information pertinent to historical events and figures[24]. Research demonstrates the prevalence of entity names in humanities users’ searches, underscoring the indispensability of NER in news analysis. Moreover, NER not only aids in document indexing but also benefits downstream processes like event detection. Additionally, it plays a crucial role in data analysis and visualization, facilitating entity-based semantic indexing and cross-linking of heterogeneous heritage collections. However, recognizing and classifying NEs in News historical texts presents unique challenges due to domain heterogeneity, input noisiness, and the dynamics of language. Despite these challenges, NER remains integral to unlocking the wealth of knowledge encapsulated within historical documents, thereby driving advancements in historical research and digital humanities[25].

The authors highlight the significance of knowledge graphs in education, scientific research, social media, and medical care. They also emphasize the

pivotal role of knowledge graphs in AI systems, including recommender systems, question-answering systems, and information retrieval tools[26]. The paper acknowledges the technical limitations and challenges associated with knowledge graph technologies, including knowledge graph embeddings, knowledge acquisition, knowledge graph completion, knowledge fusion, and knowledge reasoning. Overall, the paper serves as a valuable resource for researchers, practitioners, and stakeholders interested in understanding the multifaceted landscape of knowledge graphs. Existing models have addressed the challenge of capturing hierarchical relationships within knowledge graphs. Methods for embedding entities and categories, utilizing clustering algorithms, and incorporating type information into knowledge graph embeddings have been proposed[27], [28].

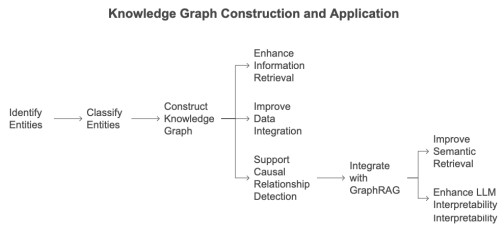


Fig. 4: Knowledge Graph extraction from news articles showing relationships between entities such as people, organizations, and locations.

D. Knowledge Graphs Representation for Event-Related News

The paper ”Knowledge Graphs Representation for Event-Related E-News Articles”[29] provides a literature review on the topic of knowledge graphs (KGs) and their applications in representing information from e-news articles. The authors begin by highlighting the challenges faced by e-newspaper readers when dealing with massive texts on e-news articles. They note that current automatic text summarizing methods are based on traditional approaches that extract only the keywords or key sentences in a text document, giving more importance to the words that are more frequent in the text document.

The authors then discuss how machine learning approaches have improved presentation accuracy. They note that while the general sequence-to-sequence text extraction methods have significant improvement compared to the traditional methods, they cannot be easily accommodated with the contextual information of a news event, such as background information that provides a broader understanding of an event, person, or item in a news document to obtain higher-level abstraction.

To address these limitations, the authors propose a solution using knowledge graphs. They provide a comprehensive review of existing KGs, including ConceptNet, Freebase, DBpedia, YAGO, BabelNet, and NELL, and discuss their strengths and weaknesses. They note that traditional KGs such as ConceptNet and Freebase usually contain known facts and assertions about entities, while DBpedia is a multilingual KG that is available in 125 languages and has more than 40 million entities.

The authors then introduce their proposed KG pipeline, which builds a KG for knowledge representation in a set of e-news articles based on a user query. They guarantee the comprehensiveness and preciseness of knowledge representation by inferencing missing relational links and new relational links in e-news articles using statistical relational learning (SRL). The KG-based text representation approach presented in this paper would be beneficial for the e-newspaper readers to speedily comprehend the dominant idea of a set of e-news article collections related to the same news event.

The authors provide a comprehensive literature review on the topic of knowledge graphs and their applications in representing information from e-news articles. They highlight the limitations of current methods and propose a solution using knowledge graphs, which they demonstrate to be effective in generating comprehensive and precise knowledge representation for the corpus of e-news articles.

In NAaaS, we are using spaCy (Python Module)[30] for NER extraction from the text. Named Entity Recognition (NER) is a crucial step in constructing knowledge graphs, as it involves identifying and classifying entities mentioned in unstructured text into predefined categories such as names of persons, organizations, locations, and more. By extracting these entities, NER enables the transformation of raw textual data into structured information that can be represented in a knowledge graph. This structured representation facilitates various applications, including information retrieval, relationship extraction, and data integration. In a knowledge graph, entities identified through NER can be linked to other entities via relationships, creating a network of interconnected information that reflects real-world knowledge. This enriched data structure enhances the ability to query and analyze complex information, providing a deeper understanding and more insightful analysis of the underlying data.

E. Causal Relationship Detection and GraphRAG Integration

Causal relationship detection is a foundational task aimed at identifying cause-effect links within datasets. This module presents an integrated framework where causal relationships are extracted and represented in knowledge graphs to enhance retrieval-augmented generation (RAG) using large language models (LLMs). Leveraging GraphRAG and domain-specific causal graphs, this methodology improves semantic retrieval, inference, and interpretability of LLM outputs [31], [32].

Causal inference relies on algorithms that employ statistical techniques like conditional independence and Markov-based data splitting to support automated reasoning [33], [34]. These techniques enhance the machine understanding of how events influence one another across domains like epidemiology, economics, and journalism.

GraphRAG, which combines structured retrieval with LLM-based generation, facilitates both local (node-level) and global (graph-summary) context retrieval. This hybrid approach boosts the factual grounding and coherence of generative models [31], [35]. By integrating structured semantic data from knowledge graphs with unstructured natural language, GraphRAG significantly improves the relevance and contextuality of responses generated by language models.

In our implementation, fine-tuned NLP pipelines extract cause-effect pairs and link them to relevant events using disambiguation tools such as BLINK and EVELINK [36]. These pairs are then incorporated into an event-centric causal graph to inform LLM queries. This structured knowledge aids both timeline summarization and the tracing of complex causal chains within news streams.

While promising, challenges remain, including data quality, model interpretability, and regional bias in training corpora. Nevertheless, the integration of causal reasoning with knowledge graphs and GraphRAG sets the stage for a new generation of intelligent, explainable news analytics systems [37], [38].

III. System Architecture

Deliverable 3 As shown in Fig: 5, the architecture of News Analytics as a Service (NAaaS) is designed to seamlessly scrape news articles from various online sources, parse them to extract pertinent information, and subsequently analyze their temporal and spatial aspects. Initially, the data scraping and parsing module retrieves news articles utilizing web scraping techniques, extracting structured data such as headlines, article content, publication dates, and geographic locations mentioned within the text. This parsed data is then serialized into messages and transmitted to Kafka, acting as a message queue for further processing. Subsequently, the temporal and spatial analysis module consumes these messages from Kafka, employing Natural Language Processing (NLP) techniques to extract temporal expressions (dates, times) and spatial references (geographic locations) from the articles. These extracted details are stored within a database for subsequent analysis. Additionally, a knowledge graph generation module utilizes the stored information to create a knowledge graph representing relationships between entities mentioned in the articles, such as events, locations, and time periods. Concurrently, the Web GIS visualization component leverages the spatial information to generate interactive maps, allowing users to visualize the geographic distribution of events and trends mentioned in the articles. Collectively, this architecture offers a comprehensive solution for real-time news analysis, enabling users to gain valuable insights and explore relationships between different entities within the news ecosystem.

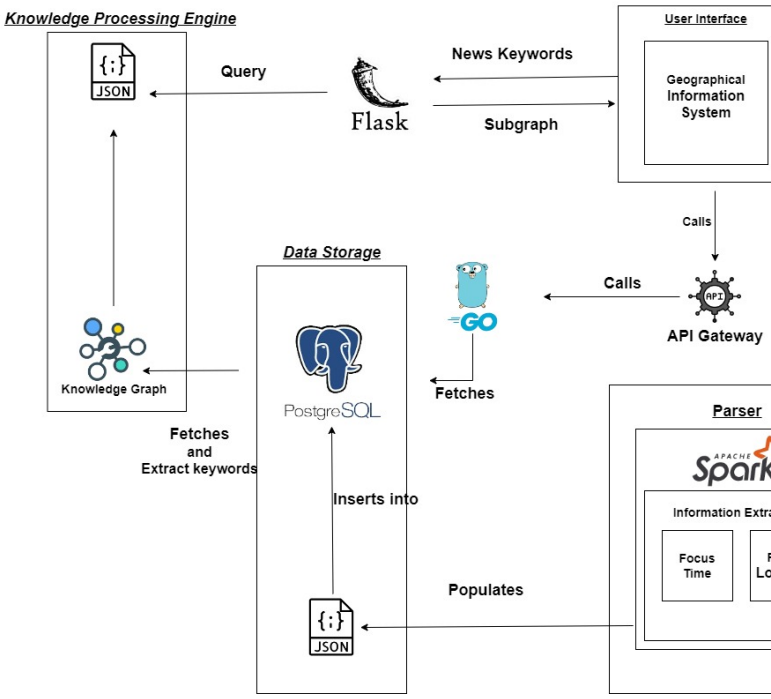


Fig. 5: Arhitecture Diagram of NAaaS Project

A. Data Acquisition via Web Scrapper

Deliverable 2

The web scraping process implemented by the Python-based Scrapper utilizes the Beautiful Soup 4 [39] library to retrieve news articles from Dawn and Tribune sources. The collected data is systematically stored within the Scrapper folder. The Scrapper is structured as follows:

1) Determine a Generic Link: Start by identifying a generic link structure for the website you want to scrape. This link might serve as the base URL for fetching news articles.
2) Define Categories of News: Identify the categories of news available on the website. These categories will help in organizing and filtering the news articles during scraping.
3) Get Previous Date: Determine the date from which you want to scrape news articles. This could be yesterday's date or any specific date you're interested in.
4) Create Links by Appending Categories and Previous Dates: Generate specific links for each category and date combination. Append the category and previous date to the generic link to create unique URLs for scraping.
5) Request to Get the Web Page: Use a web scraping library like requests to fetch the HTML content of the webpage. If the webpage is successfully retrieved, load it into a Beautiful Soup object for parsing.
    a) If Web-page is Successfully Retrieved:
        i) Extract Header, Summary, and Read More Links: Parse the HTML content to extract relevant information such as the article header, summary, and links to the full articles as shown in figure 6 and 7.
        ii) Request to Get the Full Article
        iii) If Successfully Received:
            A) Load in Beautiful Soup: Load the full article webpage into a Beautiful Soup object for further processing.
            B) Extract Data from HTML tags: Extract the full article content from the HTML structure while ensuring proper grammar and formatting.
            C) Create Header, Summary, and Article List: Organize the ex-

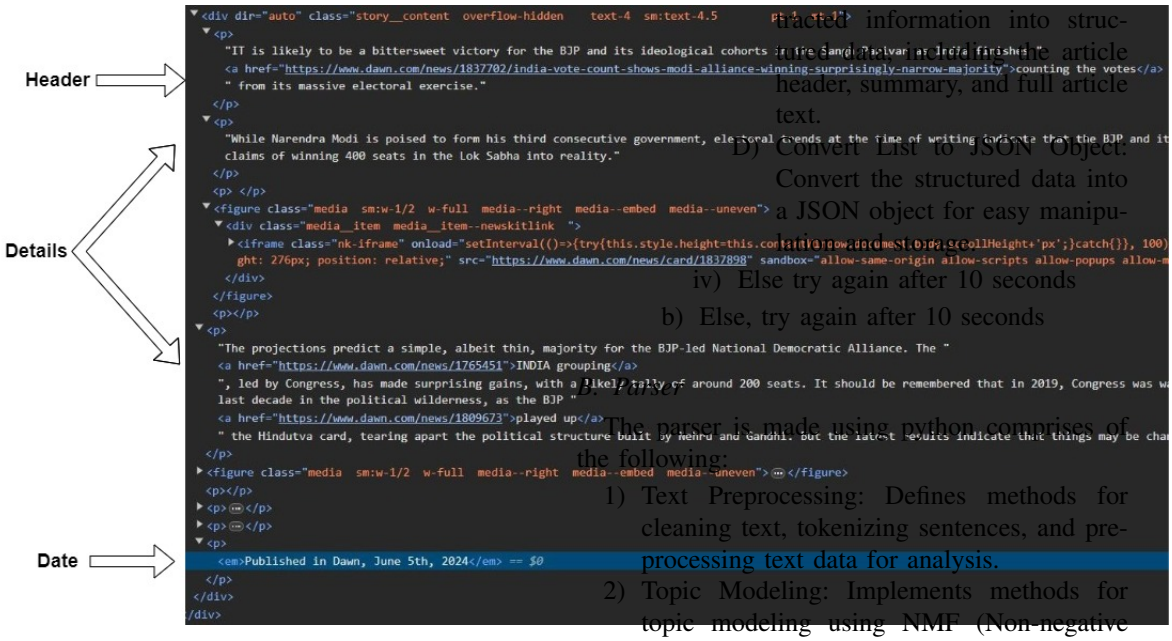


Fig. 6: HTML tags of Dawn New scrapped

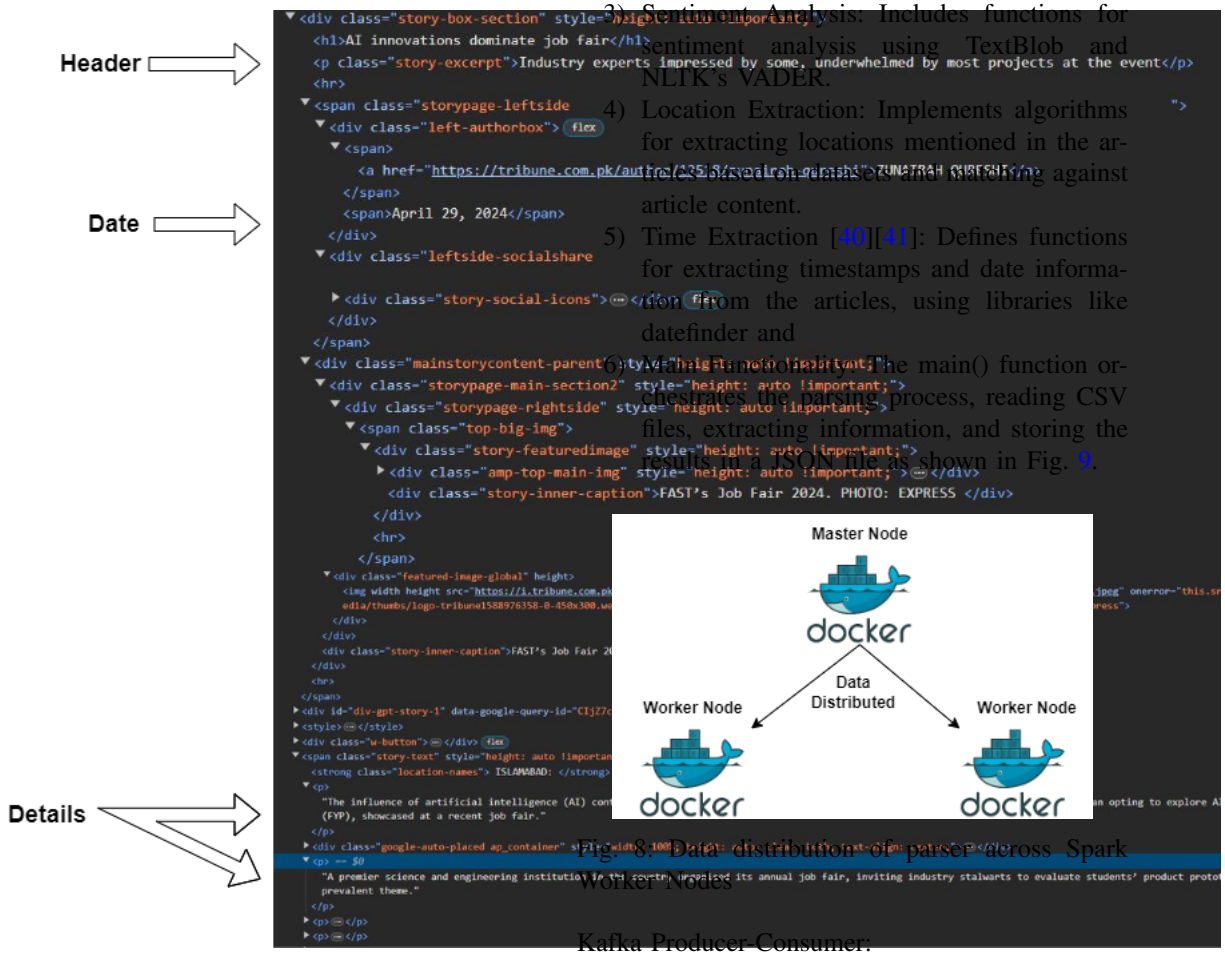


Fig. 7: HTML tags of Tribune New scrapped

tracted information into structured data, including the article header, summary, and full article text.

D) Convert List to JSON Object: Convert the structured data into a JSON object for easy manipulation and storage.

iv) Else try again after 10 seconds

b) Else, try again after 10 seconds

B. Parser

The parser is made using python comprises of the following:

1) Text Preprocessing: Defines methods for cleaning text, tokenizing sentences, and preprocessing text data for analysis.
2) Topic Modeling: Implements methods for topic modeling using NMF (Non-negative Matrix Factorization) and LDA (Latent Dirichlet Allocation).
3) Sentiment Analysis: Includes functions for sentiment analysis using TextBlob and NLTK's VADER.
4) Location Extraction: Implements algorithms for extracting locations mentioned in the articles based on datasets and matching against article content.
5) Time Extraction [40][41]: Defines functions for extracting timestamps and date information from the articles, using libraries like datefinder and
6) Main Functionality: The main() function orchestrates the parsing process, reading CSV files, extracting information, and storing the results in a JSON file, as shown in Fig. 9.

Fig. 8: Data distribution of parser across Spark Worker Nodes

Kafka Producer-Consumer:

1) A cluster of Kafka brokers was set up using docker.
2) Using Kafka-Python library, Kafka Producer class was set up connected to a kafka broker.

3) Similarly, Kafka Consumer class was set up connected to the same kafka broker as the producer.
4) Kafka-Producer gets the NEWS and stores them on topics replicated and distributed on multiple nodes.
5) Kafka-Consumer reads from those topics and sends the data to the information extraction module.


```
{
    "focusTime": "2024-01-10",
    "CreationDate": "2024-01-11",
    "Header": {
        "Text": "Poultry prices go up by 30pc"
    },
    "Summary": {
        "Text": "PPA has called for further increase in prices of chicken"
    },
    "Details": {
        "Text": "The prices of poultry items have increased by 30 per cent in the twin cities of Islamabad and Rawalpindi during the
    },
    "Link": "https://tribune.com.pk/story/2452852/poultry-prices-go-up-by-30pc",
    "Category": "islamabad",
    "focusLocation": "islamabad",
    "topics": [
        "cities",
        "increase",
        "market",
        "crisis",
        "poultry",
        "business",
        "prices"
    ],
    "sentiment": 0.5,
    "picture": "No data"
},
```


Fig. 9: Parsed JSON Data from Parser

The summary of the acquired data is presented in Table I. The collected dataset is available at https://github.com/pcnlab/Scrapped_News_Data

The table I summarizes the acquired data from two news sources, DAWN and Tribune, covering specific time spans and providing counts for various metrics. The columns in the table include the time span, news count, keyword count, location count, district count, and tehsil count. Here is a detailed description of the table structure and its content:

- **Time Span**: The periods during which the data was collected. For both DAWN and Tribune, the data spans from May 2023 and January to April 2024.
- **News Count**: The total number of news articles collected during the specified time spans. DAWN has 2376 news articles, while Tribune has 963 news articles.
- **Keyword Count**: The total number of unique keywords extracted from the news articles. DAWN has 2597 keywords, and Tribune has 1080 keywords.
- **Location Count**: The number of unique locations mentioned in the news articles. DAWN and Tribune each have 2376 and 963 locations, respectively.
- **District Count**: The number of districts covered in the news articles. DAWN covers 1947 districts, whereas Tribune covers 564 districts.
- **Tehsil Count**: The number of tehsils (subdistricts) covered in the news articles. DAWN includes 212 tehsils, and Tribune includes 56 tehsils.

In this table:

- **DAWN** data spans from May 2023 and January to April 2024, with 2376 news articles, 2597 keywords, 2376 locations, 1947 districts, and 212 tehsils.
- **Tribune** data covers the same time periods, with 963 news articles, 1080 keywords, 963 locations, 564 districts, and 56 tehsils.

TABLE I: "Summary of Acquired Data"

	Time Span	News Count	Keyword Count	Location Count	District	Tehsil
DAWN	May 2023, January - April 2024	2376	2597	2376	1947	212

Continued on next page

TABLE I: "Summary of Acquired Data" (Continued)

Tribune	May 2023, January - April 2024	963	1080	963	564	56

C. Text Processing Engine

Deliverable 4

This module relies on Spark cluster made up of 1 master node and 2 worker nodes which distributes the data so that the parser can quickly parse the data. Figure 8 explains the data distribution between Spark cluster.

1) Temporal Specificity : Temporal data is linked to documents through various means, including the time of document creation, temporal expressions within the document text, the time of document crawling, and details regarding events or entities mentioned (such as individuals or locations).

Temporal specificity gauges the degree to which document content is tightly bound to a particular moment in time. A high level of temporal specificity indicates that the document predominantly centers around a single time point. This metric becomes crucial when a user's search intent pertains to a specific event with a clear temporal reference. In such cases, a document covering multiple events may fall short of meeting the user's information needs. In the Temporal Specificity Module, our aim is to identify the primary event time mentioned in a news report. To accomplish this, we utilize temporal taggers, leveraging the SUTime package. The process unfolds as follows:

The flowchart in Figure 11 illustrates the refined procedure for determining the temporal specificity—or focus time—of a news article in the NAaaS system. The process begins by extracting temporal tags from three segments of the article: the header, summary, and detailed body. Before this extraction, the publication date is explicitly removed from the details section to prevent it from influencing the temporal tagger.

Once the temporal tags are extracted, the system filters them to retain only those that contain valid date expressions. It then checks whether any date tags remain. If none are present, the creation date of the article is assigned as the focus time by default.

If valid date tags are found, duplicate entries are eliminated to avoid redundancy. The remaining tags are then assigned weights based on their position

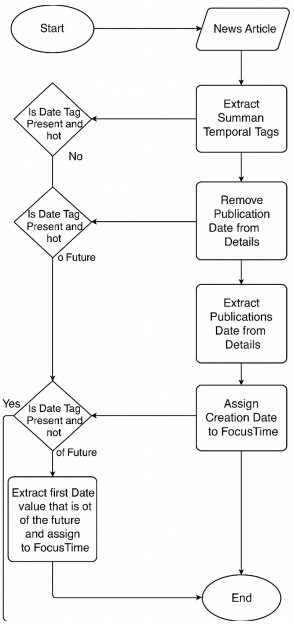


Fig. 10: Extraction of temporal specificity.

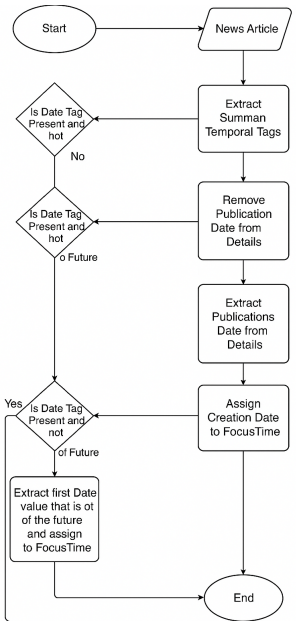


Fig. 11: Extraction of focus time.

within the article and the frequency of occurrence. For instance, dates mentioned in the headline may carry more weight than those buried in the article body. Finally, the date tag with the highest cumulative weight is selected and designated as the article's focus time.

This refined approach enhances the accuracy of temporal indexing and ensures that the extracted focus time reflects the most contextually relevant date within the article, rather than relying solely on metadata or heuristics.

2) Focus Time Extraction: We argue that focus time represents a promising avenue to tackle temporal queries, thus augmenting the effectiveness of IR systems. The determination of focus time involves analyzing the content of news documents. Given that the focus time is not explicitly stated in documents, accurately identifying it presents a significant challenge.

The temporal expression extraction procedure is illustrated in Fig: 10. The figure presents a refined flowchart that outlines the steps involved in extracting temporal expressions from news articles. The process initiates with a raw news article and proceeds through a structured sequence of temporal tag extraction operations applied to distinct parts of the article: the header, summary, and detailed content. For each section, the system checks whether a valid temporal tag exists that is not in the future. If such a tag is found, it is used as a candidate for the document's focus time. If no valid temporal expression is found in any section, the system defaults to assigning the document's creation date as the focus time.

Additionally, the chart includes a conditional step where any future-dated tags are disregarded. The procedure ensures only past or present references are considered, refining the accuracy of temporal anchoring. This robust extraction process contributes to the overall precision of timeline generation and geo-temporal indexing in the NAaaS system, thereby enhancing the effectiveness of temporal news analysis and visualization.

3) Temporal and Spatial Specificity: In the Spatial Specificity Module, our focus is on pinpointing the primary location referenced in a news report. To tackle this, we rely on pronoun taggers supported by packages such as spacy, nltk, pandas, re, and kafka. Here's how the program unfolds:

As shown in Fig. 12, we begin by fetching a JSON object from the kafka pipeline and transforming it into a structured 3D list containing the header, summary, and main body of the news article. Following this, we load a dataset sourced from the Election Commission of Pakistan, specifically focusing on areas, into a list, which is then sorted alphabetically. Moving on to the core process, we extract pronouns from the incoming news data. This involves pinpointing the start and end indexes of the areas list based on the first alphabet of the pronoun. If the first word of an area aligns with the pronoun, we extract words from the news that match the length of the pronoun.

Subsequently, we check if the extracted pronoun matches any of the listed areas. If it does, we extract words from the news before and after the

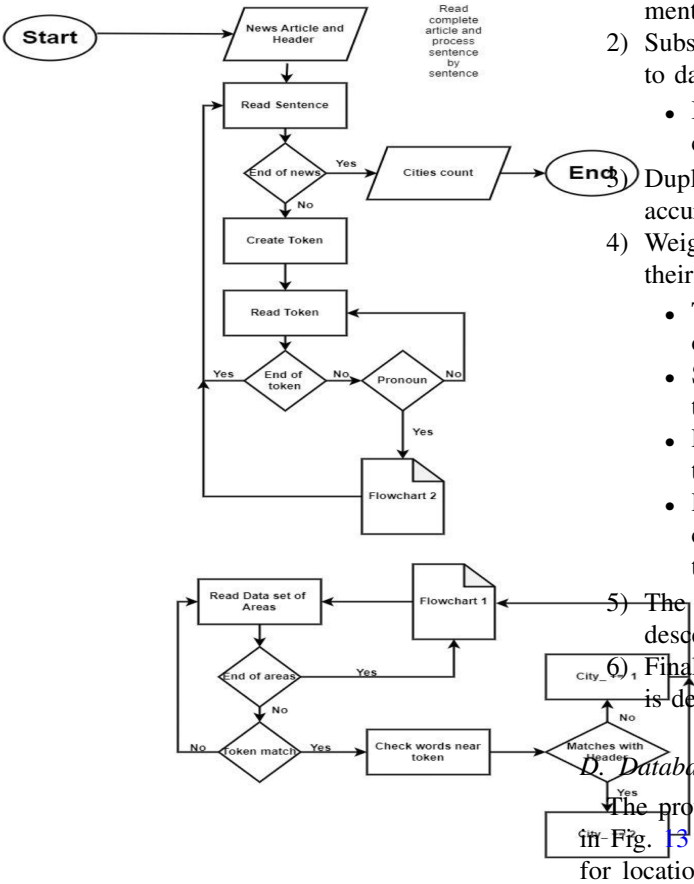


Fig. 12: Extraction of spatial location.

pronoun based on a predetermined window size. We then compare these words with the header to determine a match. If a match is found, we assign a weight to the pronoun, which consists of the normal weight plus a scaling factor. Otherwise, we assign the normal weight to the pronoun.

Throughout this process, we maintain a list of pronouns categorized as cities. Finally, we select the city with the highest weight as the focused location. This comprehensive approach ensures precise identification of the primary location discussed in the news report.

In the second iteration we updated the module to improve the accuracy of the results. For this module, we rely on temporal taggers capable of identifying time mentions within documents, employing the SUTime package for this purpose. The program follows a structured flow: For this module, we rely on temporal taggers capable of identifying time mentions within documents, employing the SUTime package for this purpose. The program follows a structured flow:

1) Initially, tags are extracted from the header, summary, and details sections of the document.
2) Subsequently, all tags except those pertaining to dates are filtered out.
   - If no tags remain after this step, the current date is assigned as the focus time.
3) Duplicate date tags are removed to ensure accuracy.
4) Weights are assigned to the tags based on their position within the document:
   - Tags from the header are given a weight of 10 times the position.
   - Summary tags receive a weight of 5 times the position.
   - Details tags are assigned a weight of 2 times the position.
   - Here, the position denotes the number of characters preceding the appearance of the date tag in the text.
5) The weighted date tags are then sorted in descending order according to their weights.
6) Finally, the date tag with the highest weight is designated as the focus time.

D. Database

The provided ER (Entity-Relationship) diagram in Fig. 13 represents a NAaas that includes tables for locations and times and visualizes them on a map. The figure shows how different entities are related and how data is structured within the system. Here's a detailed description of the diagram:

Entities and Attributes

- **Province**
  - *id*: Unique identifier for the province.
  - *name*: Name of the province.
  - *coordinates*: Geographic coordinates of the province.
- **District**
  - *id*: Unique identifier for the district.
  - *name*: Name of the district.
  - *province*: Associated province.
  - *coordinates*: Geographic coordinates of the district.
- **Union_Council**
  - *id*: Unique identifier for the union council.
  - *name*: Name of the union council.
  - *tehsil*: Sub-division or administrative unit.
  - *coordinates*: Geographic coordinates of the union council.
- **Location**
  - *id*: Unique identifier for the location.
  - *name*: Name of the location.
  - *location_type*: Type of location (province, district, etc.).

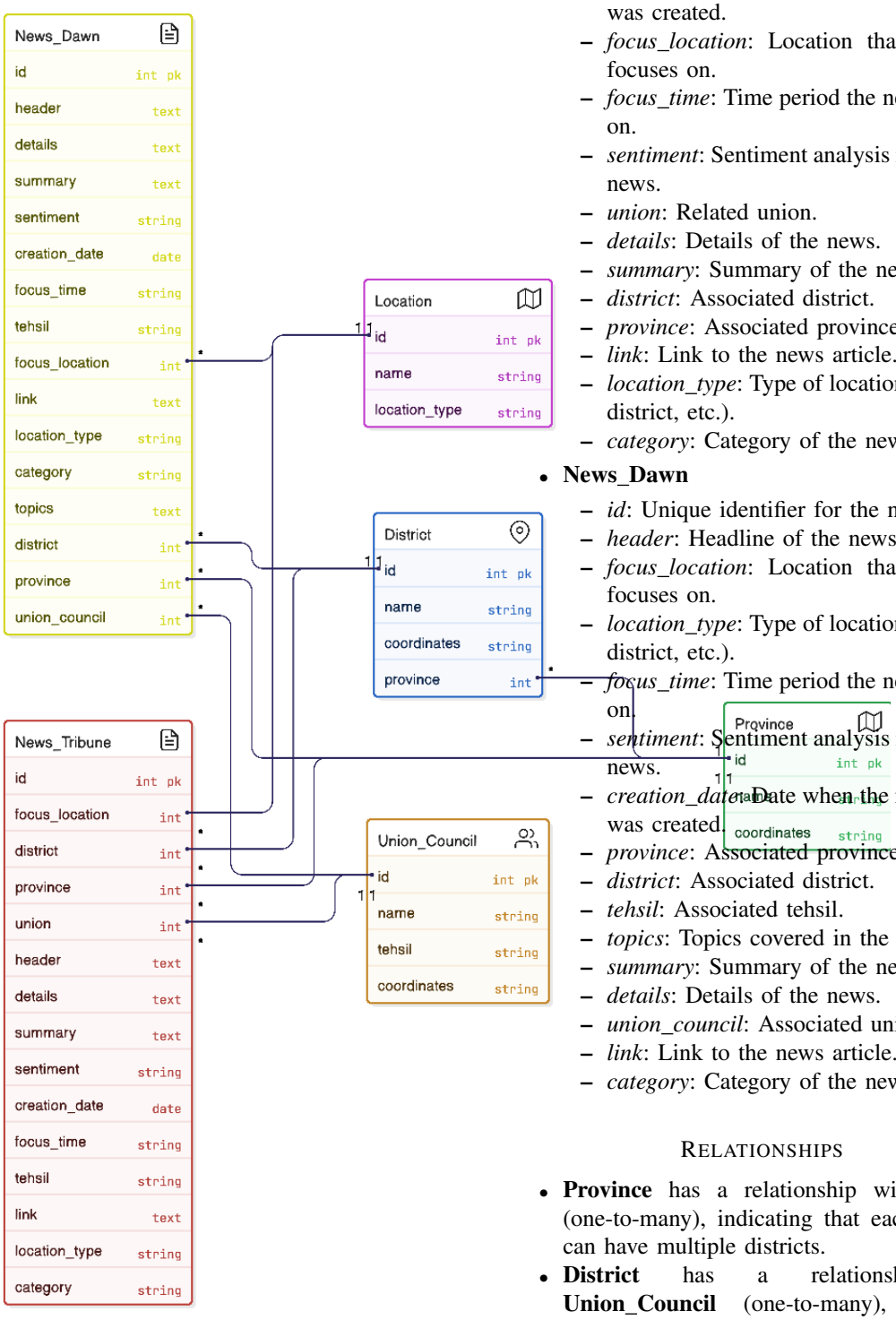


Fig. 13: Entity Relation Diagram storing NEWS in Database

- **News_Tribune**
  - *id*: Unique identifier for the news item.
  - *header*: Headline of the news article.
  - *creation_date*: Date when the news article was created.
  - *focus_location*: Location that the news focuses on.
  - *focus_time*: Time period the news focuses on.
  - *sentiment*: Sentiment analysis result of the news.
  - *union*: Related union.
  - *details*: Details of the news.
  - *summary*: Summary of the news.
  - *district*: Associated district.
  - *province*: Associated province.
  - *link*: Link to the news article.
  - *location_type*: Type of location (province, district, etc.).
  - *category*: Category of the news.
- **News_Dawn**
  - *id*: Unique identifier for the news item.
  - *header*: Headline of the news article.
  - *focus_location*: Location that the news focuses on.
  - *location_type*: Type of location (province, district, etc.).
  - *focus_time*: Time period the news focuses on.
  - *sentiment*: Sentiment analysis result of the news.
  - *creation_date*: Date when the news article was created.
  - *province*: Associated province.
  - *district*: Associated district.
  - *tehsil*: Associated tehsil.
  - *topics*: Topics covered in the news.
  - *summary*: Summary of the news.
  - *details*: Details of the news.
  - *union_council*: Associated union council.
  - *link*: Link to the news article.
  - *category*: Category of the news.

Relationships

- **Province** has a relationship with **District** (one-to-many), indicating that each province can have multiple districts.
- **District** has a relationship with **Union_Council** (one-to-many), indicating that each district can have multiple union councils.
- **Union_Council** has a relationship with **Location** (one-to-many), indicating that each union council can have multiple locations.
- **Province** has a relationship with **Location** (one-to-many), indicating that each province can have multiple locations.
- **News_Tribune** and **News_Dawn** are related to **Province**, **District**, and **Union_Council**

through multiple relationships, indicating that news articles can be associated with different administrative levels and locations.
- **News_Tribune** and **News_Dawn** contain various attributes related to news content, including focus location, focus time, sentiment, creation date, and more, providing detailed information about the news articles and their geotemporal context.

Database Schema Description

The database schema is designed to store and manage geographic and news-related data, integrating multiple sources and administrative divisions in a normalized structure. The key entities in the schema include **Province**, **District**, **Union Council**, **Location**, **News_Tribune**, and **News_Dawn**. The relationships among these entities enable effective mapping of news articles to specific administrative regions and locations.

- **Province**: Represents the highest-level administrative division. Each province has a unique identifier, a name, and geographic coordinates. A province contains multiple districts.
- **District**: Each district belongs to a province and is identified by a unique ID, a name, and coordinates. This structure supports a hierarchical relationship between provinces and districts.
- **Union Council**: This is a lower-level administrative unit, which includes an ID, name, tehsil (sub-district), and coordinates. It can be associated with multiple news items to indicate localized reporting.
- **Location**: A generic entity used to represent the focus location of a news item. It includes an ID, name, and a location type (e.g., city, village, or union council), allowing flexibility in how geographic focus is represented.
- **News_Tribune**: This entity stores news articles sourced from the Tribune. Each article includes attributes such as header, summary, details, sentiment, creation date, category, link, and metadata indicating its geographic context (province, district, tehsil, union, location type, and focus location). It also includes temporal information like focus time.
- **News_Dawn**: Similar to *News_Tribune*, this entity stores articles from Dawn. It captures the same set of attributes, with an additional field for topics to provide thematic classification.

The schema captures relationships such as:

- Provinces *have* districts.
- Districts and provinces *contain news*.

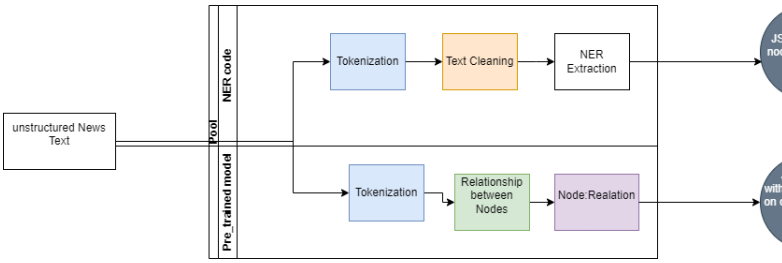


Fig. 14: Flow of text to model and NER Extraction

- Union councils *provide news* and *have news of* a location.
- News articles are linked to geographic entities (province, district, union council) and to specific locations they focus on.

This design supports comprehensive analysis of news distribution across regions and over time, while maintaining traceability to original data sources and administrative units.

E. Knowledge Processing Engine

NAaas integrates advanced knowledge processing capabilities to unlock valuable insights from news articles. At its core lies a sophisticated knowledge processing engine designed to extract Named Entity Recognition (NER) from the textual content. Leveraging cutting-edge natural language processing techniques, this engine identifies and classifies entities such as Persons, Organizations, and Locations within news articles, laying the foundation for comprehensive analysis.

Subsequently, the service employs sentiment analysis algorithms to evaluate the tone and emotions conveyed in the articles, providing valuable context for understanding public opinion and reactions. Furthermore, the system conducts media bias analysis to assess the objectivity and fairness of news coverage, shedding light on potential biases and perspectives. By seamlessly integrating these components, the News Analytics Service empowers users to delve deeper into news articles, uncovering hidden patterns, sentiments, and biases to make informed decisions and gain valuable insights into current events and trends.

1) Knowledge Graph Creation: We are using Knowledge Graph as part of a project for the NAAS to analyze news articles and extract meaningful insights and relationships.Knowledge Graph leverages natural language processing (NLP) techniques using the `spacy` library to perform tasks such as named entity recognition (NER), syntactic parsing, and keyword extraction.

The `NewsMining` class handles the entire workflow, from cleaning and preprocessing the text to extracting subject-verb-object (SVO) triples and constructing co-occurrence matrices of entities. Custom modules like `GraphShow` and `TextRank` are used for graph visualization and keyword extraction, respectively. The main function processes the articles, extracts relevant information, and generates JSON files for further analysis. This comprehensive approach enables the identification of key entities, their relationships, and significant events within the text, supporting NAAS objectives in data-driven decision-making and information extraction.

Fig: 14 illustrates the that Text passing to both model and NER extraction module.After passing text to both module ,both module preprocess the text.Then NER module get NER from text and generate JSON file also the pretained model generate the JSON file which have realtion of each entity to other .Final Calculate the difference between both JSON files and generate a final JSON file.[42], [43], [44]

Keyword Extraction with TextRank The algorithm utilizes the TextRank algorithm for keyword extraction. TextRank is a graph-based ranking algorithm inspired by Google's PageRank, where each word in the document forms a vertex in a graph, and the edges represent the co-occurrence of words within a certain window. By iteratively updating the rank scores of words based on the similarity of their contexts, TextRank identifies keywords that are central to the document's overall meaning.
Keywords extracted using TextRank serve as important indicators of the main themes and topics discussed in the news articles. These keywords provide valuable insights into the content of the articles and help in summarization, categorization, and information retrieval tasks.

Named Entity Recognition (NER) Named Entity Recognition is performed using the spaCy library, a leading NLP toolkit. The code identifies entities such as persons, organizations, and locations within the text by leveraging pre-trained statistical models and neural network architectures.

Text Cleaning Methods

- `clean_spaces(self, s)`: Removes unwanted spaces and newline characters from text.
- `remove_noisy(self, content)`: Removes text within brackets to clean up the content.

Entity and Syntax Parsing Methods

- `collect_ners(self, ents)`: Collects named entities of specified types (PERSON, ORG, GPE) from text.
- `conll_syntax(self, sent)`: Converts a sentence to CoNLL format, useful for syntactic parsing.
- `syntax_parse(self, sent)`: Parses a sentence and returns syntactic dependencies.
- `build_parse_chile_dict(self, sent, tuples)`: Builds a dictionary of child nodes for each word in a sentence.
- `complete_VOB(self, verb, child_dict_list)`: Finds the direct object (VOB) for a given verb.
- `extract_triples(self, sent)`: Extracts subject-verb-object (SVO) triples from a sentence.
- `extract_keywords(self, words_postags)`: Extracts keywords from text using TextRank.
- `collect_coexist(self, ner_sents, ners)`: Constructs NER co-occurrence matrices.
- `combination(self, a)`: Generates all possible combinations of elements in a list.

Main Function The `main` function processes a list of news article contents. It performs text cleaning, sentence splitting, NER extraction, SVO triple extraction, keyword extraction, and co-occurrence analysis. It then saves the results into JSON files.

- **Initialize Variables**: Various lists and dictionaries are initialized to store tokens, POS tags, NER entities, triples, and events.
- **Text Cleaning**: Each article content is cleaned to remove line breaks, tabs, and bracketed text.
- **Sentence Splitting**: The cleaned content is split into sentences using the `spacy` model.
- **POS Tagging and NER Extraction**: Each sentence is tokenized, POS tagged, and NER entities are extracted.
- **SVO Triple Extraction**: Subject-verb-object triples are extracted from sentences containing specified NER entities.
- **Keyword Extraction**: Keywords are extracted from the text using TextRank.
- **Event Detection**: Events are detected based on the presence of keywords in the extracted triples.
- **NER Co-occurrence**: NER co-occurrence matrices are constructed to identify relationships between entities.
- **Data Filtering**: The extracted events are filtered and refined based on various criteria.
- **Graph Construction**: A graph of the extracted information is constructed and saved as a JSON file.

Significance

NER plays a crucial role in identifying and categorizing important entities mentioned in the

news articles. By extracting entities like persons, organizations, and locations, the code enables the identification of key actors, events, and geographical locations discussed in the articles. This information is essential for understanding the context, relationships, and significance of various entities mentioned in the text.The code used for implementing these techniques can be found at: GitHub

Overall, the combination of keyword extraction using TextRank and NER using spaCy enhances the code's capability to automatically identify and extract relevant information from news articles, facilitating tasks such as event detection, summarization, trend analysis, and information retrieval. These techniques are instrumental in transforming unstructured textual data into structured information, supporting decision-making processes and enabling deeper insights into the content of news articles.

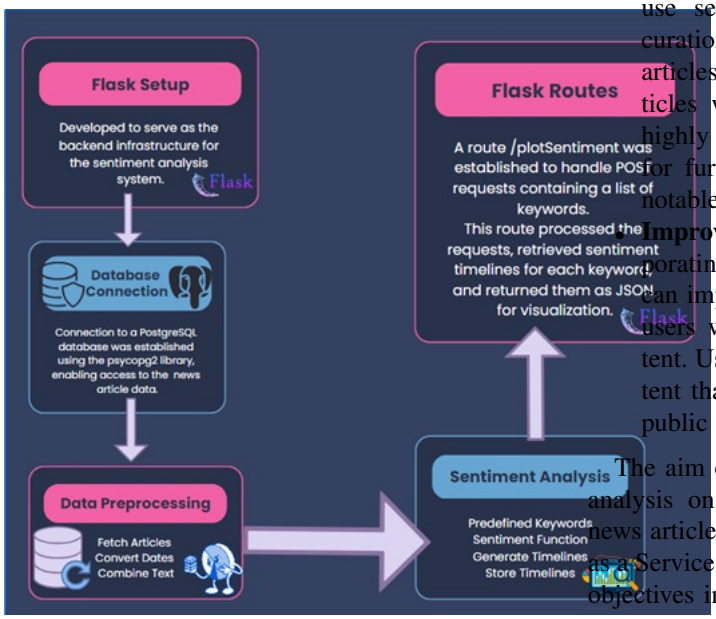


Fig. 15: Sentiment Analysis Flow chart

F. Sentiment Analysis

Sentiment analysis is a powerful technique used to understand the emotional tone conveyed in text, such as news articles or social media posts. By analyzing sentiment timelines, decision-makers can gain actionable insights into public opinion, facilitating informed decision-making processes across different domains. This report delves into the sentiment analysis performed on news articles using Python and Flask, providing insights into the sentiment trends associated with various keywords.

The aspect analysis module plays a crucial role in News Analytics as a Service (NAAS) by providing valuable insights into the sentiment expressed in news articles. This sentiment analysis is important for several reasons:

- **Understanding Public Opinion :** By analyzing sentiment, NAAS can understand public opinion and sentiment towards various topics, events, or entities mentioned in news articles. This understanding helps in gauging public sentiment towards political figures (e.g., "PPP", "Khan"), providing insights for policymakers and decision-makers.
- **Identifying Trends and Patterns :** Sentiment analysis allows NAAS to identify trends and patterns in public sentiment over time. By creating sentimental timelines, NAAS can track sentiment fluctuations and identify significant events or developments that influence public opinion.
- **Enhancing Content Curation :** NAAS can use sentiment analysis to enhance content curation by prioritizing or highlighting news articles with significant sentiment polarity. Articles with extreme sentiment scores (either highly positive or negative) may be prioritized for further analysis or presented to users as notable or trending topics.
- **Improving User Experience :** By incorporating sentiment analysis insights, NAAS can improve the user experience by providing users with more relevant and engaging content. Users can benefit from curated news content that reflects the prevailing sentiment and public opinion on various topics of interest.

The aim of this module is to perform sentiment analysis on text data using keywords related to news articles within the context of News Analytics as a Service (NAAS) as shown in Fig: 15. The main objectives include:

- Analyzing the sentiment conveyed in news articles to determine whether they are positive, negative, or neutral.
- Assigning a numerical polarity score to each article based on its sentiment, with positive values indicating positive sentiment, negative values indicating negative sentiment, and zero indicating neutral sentiment.
- Conducting sentiment analysis for specific keywords of interest such as "Voters", "PPP", and "Khan".
- Identifying and analyzing sentiment trends associated with these keywords to gain insights into public opinion and attitudes towards specific topics or entities.
- Creating sentiment timelines to visualize the sentiment trends over time for each keyword.

- Organizing sentiment data daily to track how sentiment changes over time, providing a comprehensive view of sentiment fluctuations.

Fig: 16 illustrates the systematic workflow for analyzing sentiment from news articles using a web-based software application. The system is developed using the `Flask` framework, which serves as the backbone for managing web routes and user interactions. Upon initialization, a connection is established with a PostgreSQL database via the `psycopg2` library, enabling efficient access to the repository of news articles.

During the data preparation phase, articles are retrieved from the `news_dawn` table and structured into a `Pandas DataFrame`. Key textual fields—namely, `header`, `summary`, and `details`—are concatenated into a unified field called `CombinedText`, thereby creating a comprehensive textual representation for each article.

The system then prompts the user to input between one and three keywords. It validates the keyword count and checks whether the specified terms exist in the database. If the input is valid, sentiment analysis is performed using the `TextBlob` library, which computes the polarity of the text to determine whether it reflects a positive, negative, or neutral sentiment.

Before visualizing the sentiment, the system extracts the `focus_time` (publication date) associated with each keyword occurrence and sorts these chronologically. This step enables the creation of accurate, time-aware sentiment timelines. Sentiment scores are averaged per date to illustrate trends over time.

Finally, `Flask` routes are configured to manage incoming requests and return sentiment results in JSON format, supporting interactive visualizations on the client side. Overall, this architecture provides a robust and modular framework for extracting, analyzing, and visualizing sentiment dynamics from textual news data.

G. Media Bias Detection

Media plays a big role in shaping what people think. When the media is biased, it can lead people in the wrong direction, so it's important to identify and expose this bias.

However, current methods for detecting media bias often fall short. These methods rely heavily on basic word patterns, but they struggle with new events where words are used in different ways, making it hard to spot bias.

Our goal is to find, understand, and highlight biases in a collection of news articles, even when the biases are subtle or related to new topics.

1) Types of Bias::

- **Word-based Bias :** Typically relies on analyzing the frequency and sentiment of specific words and phrases. It can be done using simple text analysis techniques and can be effective in identifying overt biases in language.[45] [46] [47]
  - **Challenge** In news articles, words can be used in different ways depending on the story. For example, the word "green" might mean environmentally friendly in one article and inexperience in another. This makes it tricky to spot bias because the meaning of words can change based on the context they are used in. So, if we only look at the words themselves, we might miss the bigger picture of how they are being used.
  - **Benefit** It provides a fast, efficient, and objective way to spot overt bias, making it useful for large-scale initial screenings and quick analysis.
- **Context-based Bias:** Requires a deeper understanding of the article's content, the relationships between different pieces of information, and the broader narrative. This type of bias is harder to detect because it involves more nuanced analysis, including understanding what information is omitted or emphasized, the framing of the story, and the selection of sources and images. [48] [48] [49]
  - **Challenge** It's covers how the whole story is told, what details are included or left out, who is quoted, what images are shown, and how the story is framed. To understand this kind of bias, we need more than just basic tools. We need sophisticated analysis methods and often human judgment to see the full picture and understand the underlined meanings. This is much harder than just looking at individual words.
  - **Benefit** It offers an in-depth, refined understanding of bias by examining the full narrative, which is essential for detecting subtle and complex biases.

2) Challenges::

- Media bias covers a great domain as there are different types of bias such as[50]:
  - **Political Bias:** Political bias occurs when media outlets show partiality towards a specific political party, ideology, or candidate. It can manifest in the selection of news stories, the framing of issues, and the emphasis placed on certain perspec-

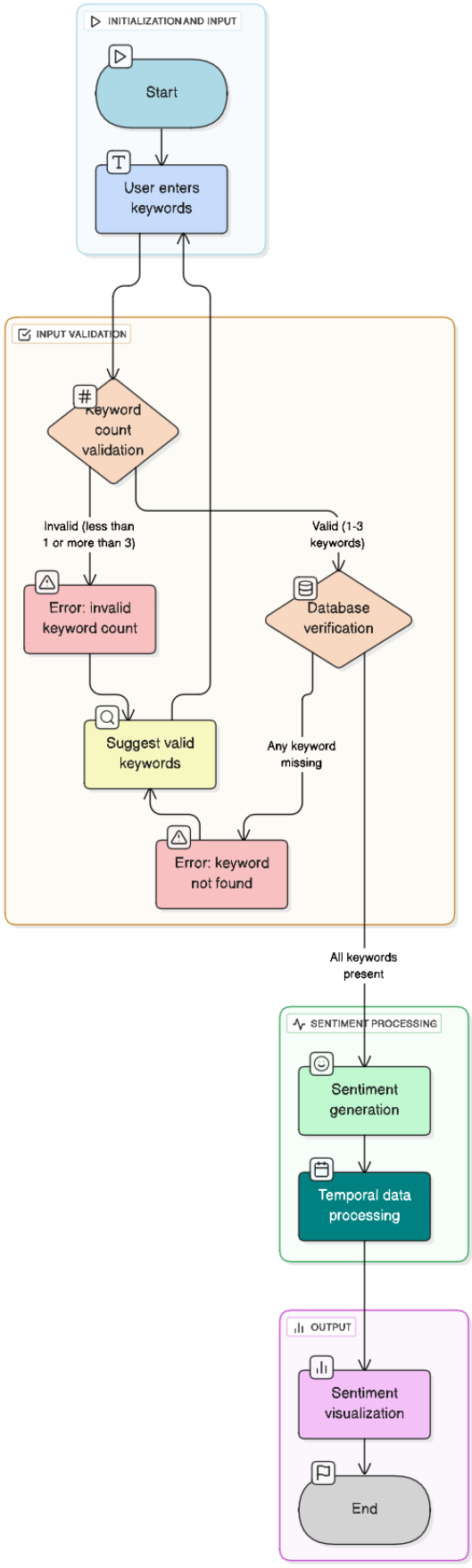


Fig. 16: Flow chart for Sentiment Extraction

tives. For example, a news organization might consistently highlight the positive actions of one political party while downplaying or criticizing those of another.
  - **Racial Bias:** Racial bias in media refers to the portrayal of events, issues, or individuals based on racial stereotypes or prejudices. This can include underrepresentation or misrepresentation of certain racial groups, perpetuating negative stereotypes, or giving preferential coverage to stories involving particular races. This bias can influence public perceptions and reinforce societal inequalities.
  - **Corporate Bias:** Corporate bias arises when media coverage is influenced by the interests of corporate owners or advertisers. News outlets might avoid stories that could negatively impact their sponsors or parent companies, or they may promote content that aligns with the financial interests of these entities. This can lead to a lack of critical reporting on corporate practices or economic issues.
  - **Cultural Bias:** Cultural bias occurs when media content favors the customs, values, or perspectives of a particular culture over others. This can result in the marginalization or misrepresentation of minority cultures. For example, news coverage may focus predominantly on Western viewpoints and lifestyles, ignoring or misinterpreting the experiences and perspectives of non-Western cultures.
  - **Omission Bias:** Omission bias involves the deliberate or inadvertent exclusion of certain facts or perspectives in media reporting. This can lead to an incomplete or skewed understanding of an issue. For instance, a news story might leave out critical information that could alter the audience's perception of the event or topic being reported, thereby influencing public opinion by what is not said rather than what is said.
- There are models designed to detect these types of biases in news reporting. However, these models often rely on datasets primarily composed of American news sources. As a result, they lack diverse data from different regions and cultures, which limits their effectiveness and accuracy when applied globally.
- Before we can analyze the text with our model, it needs to be broken down into smaller pieces called tokens. However, many of the articles produce too many tokens, exceeding the limit that our model can handle.
- After running the model on articles, we got some incorrect results (false positives). This happened because the model was mainly trained on American news sources.

3) Activities:: To determine if an article is biased or unbiased, we used a classification model. Here's a breakdown of which model we used and the type of tool it required:

- **ConvBert:**This model analyzes text using a special tool called ConvBertTokenizer.
- **BART:**Another model we used, which requires a different tool called BartTokenizer.
- **GPT-2:**This model uses yet another tool called GPT2Tokenizer.
- **Electra:**For this model, we needed a tool called ElectraTokenizer.
- **RoBERTa:**Lastly, there's RoBERTa, which requires a tool called RobertaTokenizer.

Each model was used to find a specific type of bias such as:

- **Hate Speech:** This is when an article unfairly criticizes people because of their race or religion. It's like someone saying mean things about others just because of who they are.
- **Gender Bias:**Sometimes, articles can make one gender seem better or worse than the other. This tool helped us spot when articles were doing that unfairly.
- **Racial Bias:** Imagine if an article kept saying bad things about people of a certain race, without any good reason. That's what this tool helped us find.
- **Political Bias:**Have you ever seen an article that seems to really like one political party and doesn't say much good about the others? That's what this tool looked for – when articles show unfair support for one side without giving a fair chance to others.

H. Web-GIS Formation

Deliverable 5 At the final stage, the News Analytics Service utilizes a Web-based Geographic Information System (GIS) to visualize and display the extracted insights from the news articles. This Web-GIS system serves as a dynamic platform for spatially tagging the news articles at their relevant geographical focus locations, offering users an interactive and intuitive interface to explore the geographical context of the news content. Leveraging powerful mapping capabilities, the Web-GIS system overlays the tagged news articles onto interactive maps, allowing users to visualize the distribution and spatial patterns of news events across different regions. Through this immersive visual

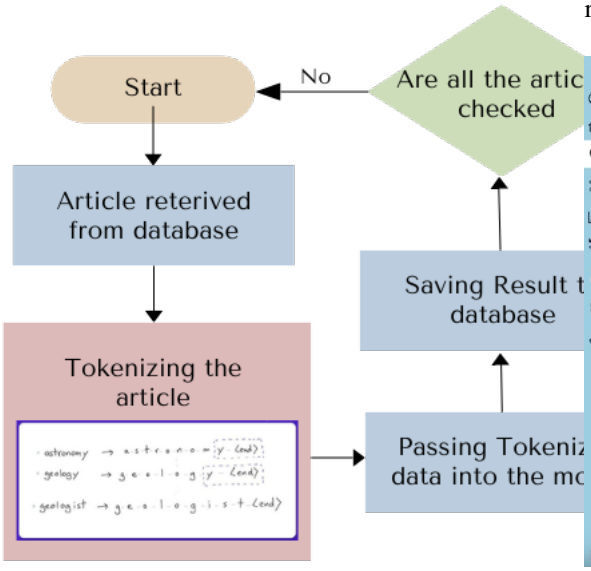


Fig. 17: Media Bias Flow chart

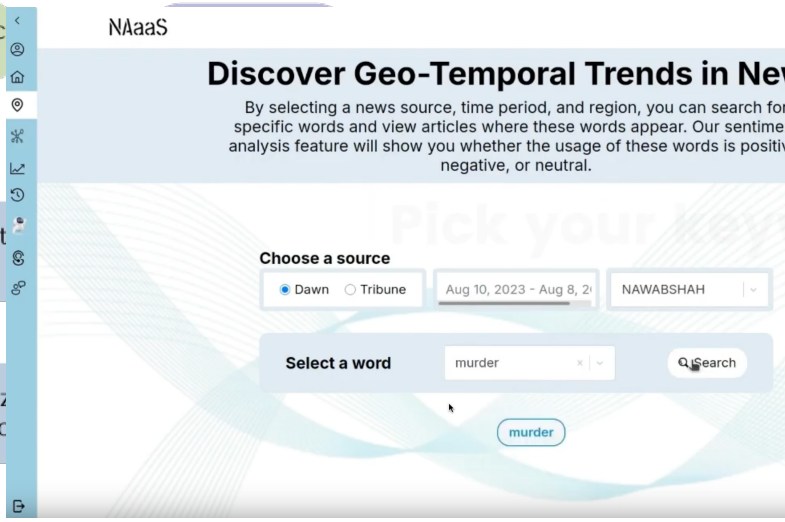


Fig. 18: Homepage of the NAAAS (News Analytics as a Service) tool

representation, users can gain a deeper understanding of the geographical contexts surrounding news events, identify spatial trends, and uncover insights that may not be apparent from textual analysis alone. By harnessing the capabilities of Web-GIS technology, the News Analytics Service enhances the accessibility and usability of its insights, enabling users to make informed decisions based on a spatial understanding of current events and trends.

The interface of Web-GIS has been shown in Fig; 20. User will select the source of the news along with time-range of the focus time of the news items. Once the news will be extracted based on the search criteria, keywords list will be populated and user can further shortlist the news by selecting multiple keywords simultaneously.

1) Interactive Visualization: Interactive and user-friendly visualizations are developed for the GIS system, allowing users to filter and explore data by time, location, and keyword. Tools are incorporated for users to drill down into specific events, locations, or entities to gain deeper insights as shown in the figures showing results. The interface displayed in the screenshot is the main landing page of the NAAAS (News Analytics as a Service) application, designed to help users discover geo-temporal trends in news articles. The layout is clean, intuitive, and user-friendly, allowing users to interact with the system efficiently. At the center of the interface, users are prompted to begin by selecting a news source—either Dawn or Tribune. This is followed by options to specify a time period and a geographical region, enabling users to filter news content based on temporal and spatial relevance.

a) Description:: Figure 18 displays the homepage interface of the NAAAS platform, which is designed to allow users to explore geo-temporal trends in news data. At the top, the interface prominently features the system title **NAAAS** and a headline, “Discover Geo-Temporal Trends in News,” followed by a brief description of the system’s functionality. The system enables users to select a news source, specify a time period and region, and search for specific keywords within news articles. Additionally, it provides sentiment analysis to assess whether the usage of searched terms in news content is positive, negative, or neutral.

The interface includes the following interactive elements:

- **Source Selection:** Radio buttons to choose between available news sources, such as *Dawn*, *Geo News* and *Tribune*.
- **Time and Region Filters:** Dropdown menus labeled “Choose Time” and “Choose Region” to allow temporal and spatial filtering of news data.
- **Keyword Input:** A text box with the placeholder “i.e. police, protest” enabling the user to enter terms of interest.
- **Search Button:** A blue button labeled “Search” that initiates the query based on selected filters and entered keywords.

The sidebar on the left contains icons for navigating between different modules of the platform, although their functions are not labeled in this screenshot. At the bottom right, a chatbot icon is visible, providing help and support to users. This

intuitive UI layout facilitates efficient interaction with the system's advanced analytics backend described in the architecture.

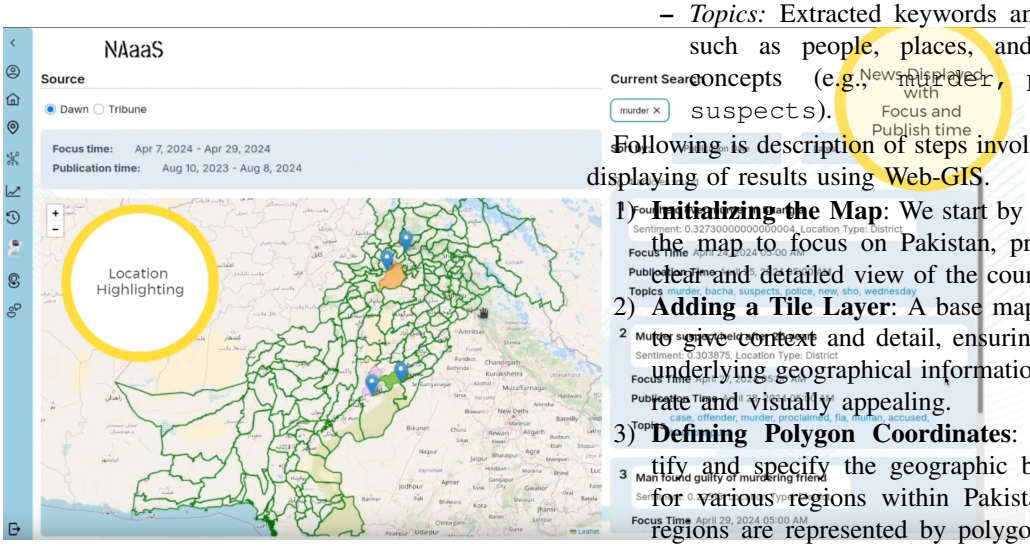


Fig. 19: Geo-temporal News Search Results in the NAAAS System

Figure 19 illustrates the output interface of the NAAAS tool following a user's query. After the user selects the data source, time frame, region, and a keyword (as shown in Figure 18), the system returns a filtered set of relevant news articles based on the specified criteria.

The interface is divided into three main panels:

- **Left Panel:** Shows a map of Pakistan with highlighted regions. Colored markers and polygons are used to indicate locations referenced in the retrieved articles. This spatial visualization aids in identifying geographical concentration and distribution of events.
- **Top Center Panel:** Displays selected metadata including:
  - *Source:* e.g., Dawn or Tribune or Geo News
  - *Focus Time:* Temporal window referring to the date range of the events discussed in the articles (e.g., April 7, 2024 – April 29, 2024)
  - *Publication Time:* The span of dates during which the relevant articles were published (e.g., August 10, 2023 – August 8, 2024)
- **Right Panel:** Lists the retrieved news articles with structured metadata, including:
  - *Title and Summary:* Headline of the news article.
  - *Sentiment Score:* A numeric value indicating the emotional tone (e.g., 0.327 for neutral to slightly negative).
  - *Focus and Publication Times:* Each article includes a precise focus date (event occurrence) and a publication timestamp.
  - *Topics:* Extracted keywords and entities such as people, places, and relevant concepts (e.g., `murder`, `police`, `suspects`).

Following is description of steps involved in the displaying of results using Web-GIS.

1) **Initializing the Map**: We start by setting up the map to focus on Pakistan, providing a clear and detailed view of the country.
2) **Adding a Tile Layer**: A base map is added to give context and detail, ensuring that the underlying geographical information is accurate and visually appealing.
3) **Defining Polygon Coordinates**: We identify and specify the geographic boundaries for various regions within Pakistan. These regions are represented by polygons on the map.
4) **Creating and Adding Polygons**: These polygons are then drawn on the map, clearly delineating the different areas we are focusing on.
5) **Adding Interactivity**: To enhance user interaction, we enable features such as clicking on a polygon to display more information about the specific region.
6) **Customizing the Map**: Various customizations are applied to improve user experience, such as custom tooltips, pop-ups, and different styles for the polygons and map interface.

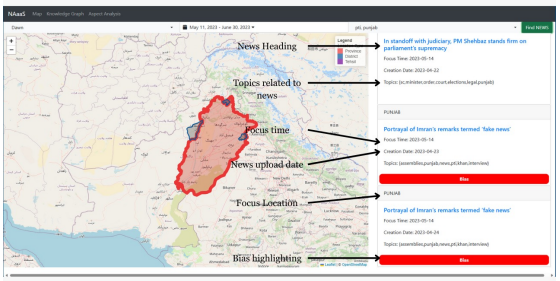


Fig. 20: Description of the Interface items

The screenshot highlights important features annotations using the geographical boundaries of the districts corresponding to the filtered news:

- **Location Highlighting:** The system marks relevant locations on the map dynamically based on article content.
- **Temporal Focus:** Articles are sorted and shown with both event (focus) time and publication time to enable temporal analysis of reporting patterns.

This visual analytics output is the result of several backend algorithms including Named Entity Recognition (NER), topic modeling, location resolution, and sentiment classification, interacting cohesively to deliver an integrated user experience. The system is designed to guide users step-by-step through the analysis process, and each input component plays a critical role in narrowing down the data to the most contextually relevant news items. This tailored approach helps users, especially those without technical backgrounds, to explore media trends and sentiment across different time frames and geographic regions with ease. The side navigation bar and help bot at the bottom-right suggest additional support and functionality, enhancing the overall usability of the application.

I. Graph RAG module for AI based News Extraction

a) System Architecture Overview: The system architecture for the proposed news analytics platform leverages modular components, enabling efficient acquisition, processing, and interpretation of news data using graph-based Retrieval-Augmented Generation (Graph RAG). As illustrated in Figure 21, the architecture comprises five primary layers: the user interaction interface, API gateway, NLP and data scraping services, semantic retrieval and community detection, and summarization with temporal reasoning. At the forefront, a user-friendly

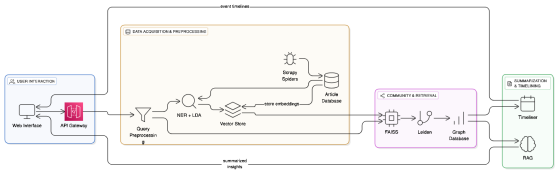


Fig. 21: System Architecture of the News Analytics Platform using Graph RAG

web interface allows users to submit queries and visualize results. The API Gateway handles these requests, performing routing and access control, and coordinates interactions with the backend services. The data scraping component employs Scrapy spiders to crawl news sources, converting raw web pages into structured content, which is then stored in the article database. Preprocessing tasks, including query normalization and Named Entity Recognition (NER) combined with Latent Dirichlet Allocation (LDA), extract key entities and topics from both user queries and news articles. These outputs are encoded as dense vector embeddings and stored in a vector store for efficient retrieval.

To retrieve relevant content, the system uses FAISS for fast approximate nearest neighbor search, retrieving the top-k most semantically similar articles. Subsequently, the Leiden algorithm is applied for community detection among retrieved entities and events, revealing clusters that signify co-occurring or related news components. These relationships are stored in a graph database, forming the backbone of the Graph RAG component.

The final layer involves reasoning and summarization. A large language model (LLM) component, integrated with a Timeliner module, generates concise summaries and chronological event narratives. This allows users to explore topic-specific sentiment evolution and entity relations over time. Altogether, this architecture enables robust and interpretable news analytics, combining symbolic reasoning with neural models to provide insight-rich outputs for decision-makers, analysts, and researchers.

IV. Experiments Undertaken

Deliverable 7

The Web-based GIS system incorporates several guidelines to effectively identify focus times and locations from news articles, plot them on a map, draw a knowledge graph using keywords, and plot sentiment trends. The system ensures that news articles are sourced from reliable and diverse outlets, capturing a broad spectrum of information. Data is thoroughly cleaned and preprocessed to remove noise and irrelevant information, improving the accuracy of subsequent analysis steps.

A. Parser Module for Information Extraction

The parser module serves as the core processing engine of the NAaaS system. It is responsible for transforming raw scrapped news articles into structured data through a comprehensive pipeline involving text cleaning, temporal tagging, topic modeling, sentiment scoring, and geo-location identification. This module operates in a distributed fashion using Apache Spark, enabling scalable processing of large news corpora.

1) Text Cleaning and Preprocessing: Each article undergoes text normalization, including lowercasing, stop word removal, punctuation stripping, and lemmatization using `spaCy` and `WordNetLemmatizer`. These steps reduce lexical variability and prepare the text for downstream NLP tasks.

2) Sentence Tokenization and Preprocessing: The module splits long articles into sentences using regular expressions to facilitate more granular analysis. It also tokenizes and filters meaningful tokens using `nltk` and standard preprocessing techniques to support both topic modeling and sentiment analysis.

3) Named Entity Recognition and Location Extraction: Advanced Named Entity Recognition (NER) models are utilized to extract relevant entities such as people, organizations, locations, and events. These models are fine-tuned with domain-specific data to improve extraction accuracy. The system focuses on extracting keywords central to the articles' themes, forming the basis of the knowledge graph and sentiment analysis.

The parser identifies geographical locations referenced in the news using a hybrid approach. It cross-references extracted named entities against a verified database of city names provided by the Election Commission of Pakistan. Fuzzy string matching (`fuzz.ratio`) is used to identify close matches, and the system assigns a weighted score based on frequency and contextual relevance to determine the most probable focus location.

4) Temporal and Spatial Analysis: Robust temporal extraction tools are used to identify and standardize date and time mentions in the articles, allowing for accurate plotting of events on the timeline. Geoparsing tools extract precise geographical locations, which are then validated against known geographical databases to ensure accuracy.

a) Temporal Tagging and Focus Time Detection: The system uses a combination of tools such as `SUTime`, `datefinder`, and `dateparser` to extract and normalize temporal expressions from the `header`, `summary`, and `details` fields of each article. A custom `TimeTag` class is employed to store date mentions along with occurrence frequency, which is then used to determine the article's focus time. If no reliable tag is found, the article's creation date is used as fallback.

5) Topic Modeling: To extract themes from articles, the parser applies both Non-negative Matrix Factorization (NMF) and Latent Dirichlet Allocation (LDA) using `scikit-learn`. Articles are vectorized with `TfidfVectorizer`, and overlapping topics from both models are used to ensure consistency and robustness in topic extraction.

6) Sentiment Analysis: The parser includes a dual-layered sentiment analyzer combining `TextBlob` and `VADER` sentiment polarity scorers. Both models independently evaluate the emotional tone of the article text, and their normalized scores are averaged to yield a final sentiment score ranging from 0 (negative) to 1 (positive).

7) Output Generation: Each processed article is summarized into a structured JSON object containing:

- **Focus Time:** The most relevant timestamp in the news content.
- **Focus Location:** The main geographical reference.
- **Topics:** Extracted themes from the article.
- **Sentiment Score:** A numerical representation of sentiment polarity.
- **Meta Information:** Header, summary, category, and link to the source.

These JSON records are saved to a file and passed to downstream components like the Web-GIS system and Knowledge Graph Generator. This structured representation enables temporal and spatial indexing of news, facilitating high-quality visual analytics.

B. Sentiment Analysis

State-of-the-art sentiment analysis models determine the sentiment of the extracted keywords, considering the context to avoid misclassification. Sentiment trends are plotted over time to identify

shifts in public opinion and correlate these trends with significant events.

1) Aspect Analysis: there are several steps involved in handling the form submission and displaying the sentiment graph:

- **Getting Keywords:** The code collects any words entered in the form's input boxes, such as 'happy' or 'sad', and stores them for later use.
- **Checking Keywords:** It ensures that at least one keyword is entered and no more than four. This step ensures that the form is properly filled out, akin to ensuring that a pizza has at least one topping but not too many to overwhelm it.
- **Sending the Keywords to the Server:** When the form is submitted, the code sends the keywords to the server, which is like sending a message to a big computer that manages the website.
- **Handling the Server's Response:** Once the server receives the keywords, it processes them and sends back a response. The code is designed to listen for this response, like waiting for acknowledgment after sending a message.
- **Creating a Graph:** Finally, based on the response received from the server, the code generates a graph or chart that visually represents the sentiment of the entered words. This graph makes it easier for users to understand the sentiment of their input, much like turning words into a colorful picture that conveys their meaning.

A function `plot_sentiment()` handles POST requests to the '/plotSentiment' endpoint of a Flask web application. Here's a breakdown of what it does:

- **Receiving Data:** When a POST request is made to the '/plotSentiment' endpoint, the function receives JSON data containing keywords sent from the client-side form.
- **Processing Keywords:** The function extracts the keywords from the JSON data and iterates over them.
- **Generating Sentiment Timelines:** For each keyword, it checks if there are articles in the DataFrame (`df`) that contain that keyword in their 'CombinedText' column. If there are matches, it calculates the sentiment timeline for that keyword by grouping articles by date and computing the average sentiment polarity for each date.
- **Creating Plot Data:** Based on the sentiment timelines generated, the function constructs plot data in the form of lists of x and y values for each keyword. These plot data points will be used to create a graph.
- **Returning Response:** Finally, the function returns a JSON response containing the plot data for visualization on the client side. If no data is available for a keyword, it prints a message indicating so.

C. Knowledge Graph Construction

The knowledge graph is constructed by identifying and linking entities mentioned in the articles, using graph databases to store and visualize these relationships. Key connections between entities are highlighted to uncover hidden relationships and patterns that might not be evident from individual articles.

D. Algorithmic Workflow for Graph-RAG-based News Analytics

The proposed system integrates multiple algorithms to support a graph-based Retrieval-Augmented Generation (RAG) pipeline for advanced news analytics. Each algorithm fulfills a distinct subtask, yet together they enable end-to-end retrieval, community detection, summarization, and question answering using news data.

a) Named Entity and Topic Integration: The process begins with Algorithm 1, which combines Named Entity Recognition (NER) with Latent Dirichlet Allocation (LDA) to extract both named entities and latent topics from a corpus of articles. Using SpaCy for NER and a topic model trained over the same articles, this algorithm builds a unified set of semantic descriptors for each article. These enriched topic representations lay the foundation for subsequent similarity and clustering operations.

b) Temporal Structuring of Content: To organize events chronologically, Algorithm 2 constructs a timeliner by comparing the semantic and named entity overlap between articles. Articles with significant intersection-over-union (IOU) in entity mentions are grouped and ordered by timestamp to provide a coherent timeline. This enhances the system's ability to surface historical narratives and event progressions.

c) Graph-based Community Detection: With entity-enriched embeddings in place, Algorithm 3 constructs a graph of article relationships, where edges represent semantic similarity and entity overlap. The Leiden community detection algorithm is applied to this graph to identify clusters of closely related articles—each representing a distinct topic community.

d) Query-to-Community Matching: When a user submits a query, Algorithm 4 determines the most relevant community by computing semantic and entity-based similarity between the query and each community. The best match, based on a weighted combination of cosine similarity and IOU, is returned for further exploration or retrieval.

e) Top-k Neighborhood Expansion: Algorithm 5 extends the retrieval to not just the most relevant communities but also their neighboring nodes in the semantic graph. This ensures the inclusion of peripheral but contextually relevant content, enhancing coverage during the RAG phase.

f) LLM-based Query Answering with RAG: Finally, Algorithm 6 implements the RAG mechanism. It aggregates content from both the core community and its neighbors, retrieves associated articles via FAISS, and compiles a context window. This compiled context is passed to a Large Language Model (LLM) to generate a coherent and contextually rich answer to the user's query.

g) Integrated System Behavior: Collectively, these algorithms orchestrate a pipeline that transforms raw news articles into a semantic graph structure rich in entity and topic information. The graph enables robust retrieval and contextual navigation, which is further enhanced by LLM-powered synthesis. By coupling symbolic community structure (via Leiden) with neural representations (via embeddings and FAISS), the architecture achieves interpretable and accurate news analytics at scale.

Algorithm 1 Named Union (NER + LDA Integration)

Require: Articles $A = \{a_1, a_2, ..., a_n\}$, SpaCy model M, Number of LDA topics k
Ensure: Combined NER and LDA topics T
Initialize $T = \{\}$, $NER\_entities = \{\}$, $LDA\_model = None$
for each article a_i in A **do**
  Run SpaCy model M on a_i
  Extract named entities and store in $NER\_entities[a_i]$
end for
Train LDA_model on A with k topics
for each document a_i in A **do**
  $Combined\_Topics[a_i] = NER\_entities[a_i] \cup LDA\_Topics[a_i]$
  Store $Combined\_Topics[a_i]$ in T
end for
return T

Algorithm 2 Creating a Timeliner

Require: Articles A, FAISS Index F, SpaCy model M, Topics k, IOU Threshold τ
Ensure: Timeliner T
Retrieve top-N articles $A_{retrieved}$ from FAISS
Initialize $NER\_entities = \{\}$, $T = [\,]$
for each a_i in $A_{retrieved}$ **do**
  Run SpaCy model and extract entities into $NER\_entities[a_i]$
end for
Train LDA on $A_{retrieved}$
for each pair (a_i, a_j) in $A_{retrieved}$ **do**
  if $IOU(NER[a_i], NER[a_j]) > \tau$ **then**
    Add a_i and a_j to T
  end if
end for
Sort T by date
return T

Algorithm 3 Creating Leiden Communities

Require: Articles A, Embeddings E, IOU threshold τ, Similarity threshold σ
Ensure: Communities C
Initialize Graph $G = (V, E)$
for each article pair (a_i, a_j) **do**
  Compute IOU and cosine similarity
  if $IOU > \tau$ and $Similarity > \sigma$ **then**
    Add edge (a_i, a_j) to G
  end if
end for
Apply Leiden Algorithm on G
return Detected communities C

Algorithm 4 Query to Community Mapping

Require: Query Q, Communities C, NER and LDA for each community
Ensure: Closest Community $c_{closest}$
Initialize $max\_similarity = -1$
for each c_i in C **do**
  Compute IOU and cosine similarity with Q
  Combine them using weighted average
  if similarity $> max\_similarity$ **then**
    Update $c_{closest}$
  end if
end for
return $c_{closest}$

Algorithm 5 Top-k Community Retrieval with Neighbors

Require: Query Q, Communities C, Graph G
Ensure: Center and Neighbor Embeddings

Run Algorithm 4 to get C_{topk}
for each c_i in C_{topk} **do**
  Add $E(c_i)$ to E_{center}
  for each neighbor n_j of c_i in G **do**
    if $IOU(Q, n_j) > \tau$ **then**
      Add $E(n_j)$ to $E_{neighbors}$
    end if
  end for
end for
return E_{center}, $E_{neighbors}$

Algorithm 6 RAG with Neighbors and LLM for Query Answering

Require: Query Q, E_{center}, $E_{neighbors}$, LLM
Ensure: Answer A

Initialize $Context = [\,]$
for each embedding e_i in $E_{center} \cup E_{neighbors}$ **do**
  Retrieve top documents using FAISS
  Extract $Text_i$ and append to $Context$
end for
Concatenate $Context$ to form $Final\_Context$
$Prompt$ = [”Given the context:”, $Final\_Context$, ”Answer the query:”, Q]
$A = LLM(Prompt)$
return A

V. Results and Discussion

We are leveraging the open-source JavaScript library Leaflet to create and display interactive maps with polygonal regions on a map of Pakistan. Leaflet provides a robust and user-friendly platform for developing mobile-friendly maps, making it an ideal choice for our project.

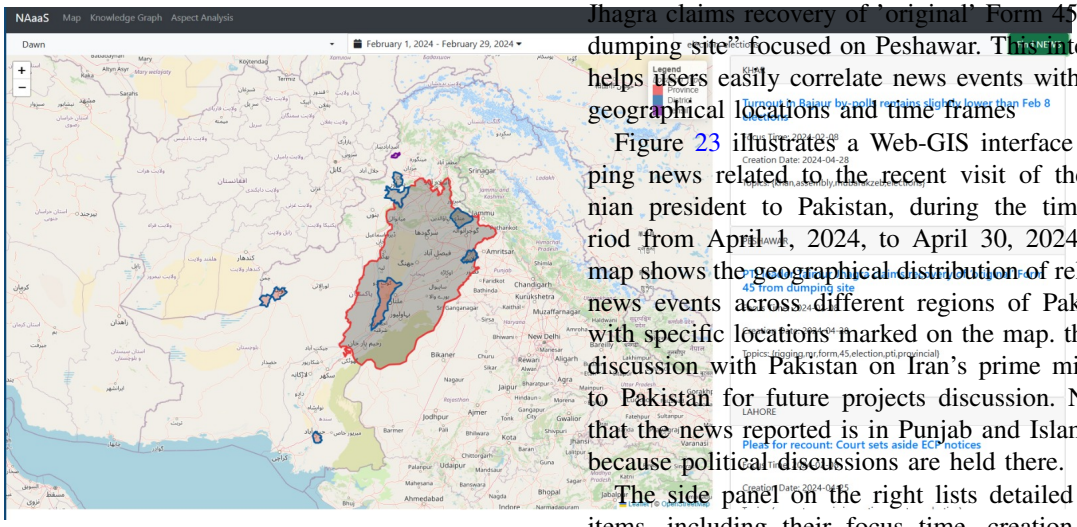


Fig. 22: Results showing election activity held in Febuary

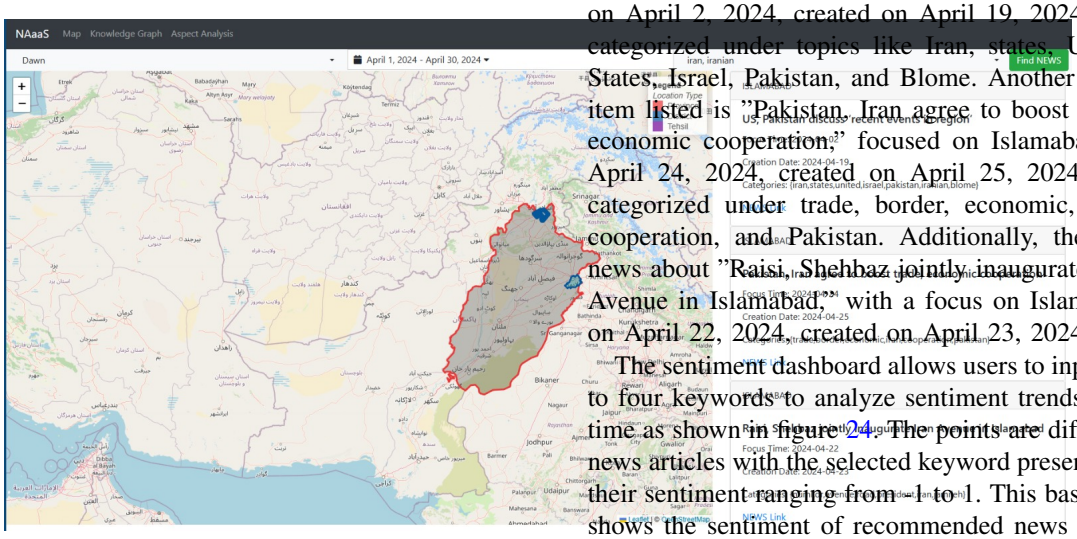


Fig. 23: Results showing US discussion with Pakistan regarding Iran issue

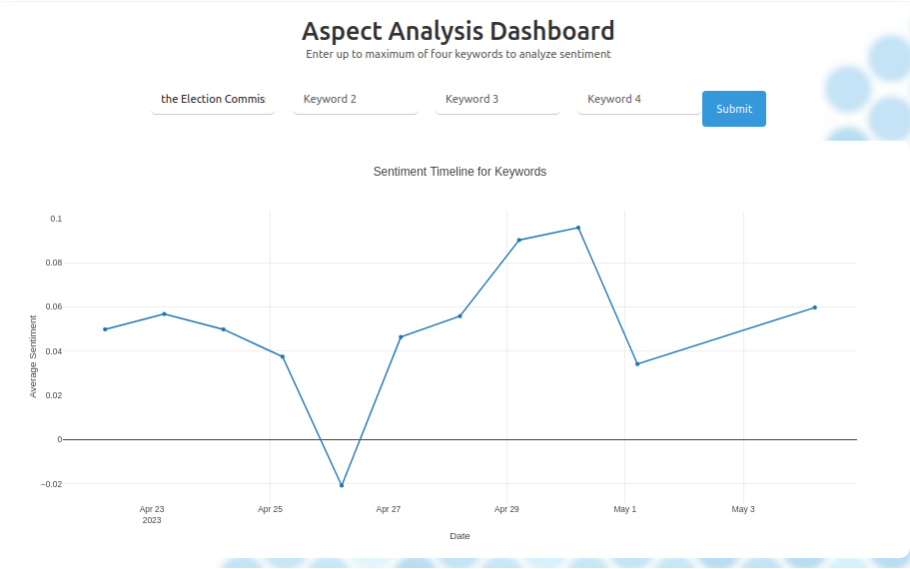
Figure 22 shows the results of elections held in Feb 2024. Notice that most activity about news is held in Punjab and only Quetta and some areas of KPK reported news activity. The figure depicts a Web-GIS interface displaying news events from the month of elections in 2024 in Pakistan. The map highlights specific geographical locations, with each highlighted area representing a province, district, or tehsil where significant news events have been reported. The side panel on the right provides a list of these news stories, including their focus time and creation date, as well as associated topics. For instance, the panel mentions news such as "Turnout in Bajaur by-polls remains slightly lower than Feb 8 elections" with a focus on Khar, and another report about "PTI leader Taimur Jhagra claims recovery of 'original' Form 45 from dumping site" focused on Peshawar. This interface helps users easily correlate news events with their geographical locations and time frames

Figure 23 illustrates a Web-GIS interface mapping news related to the recent visit of the Iranian president to Pakistan, during the time period from April 1, 2024, to April 30, 2024. The map shows the geographical distribution of relevant news events across different regions of Pakistan, with specific locations marked on the map. the US discussion with Pakistan on Iran's prime minister to Pakistan for future projects discussion. Notice that the news reported is in Punjab and Islamabad because political discussions are held there.

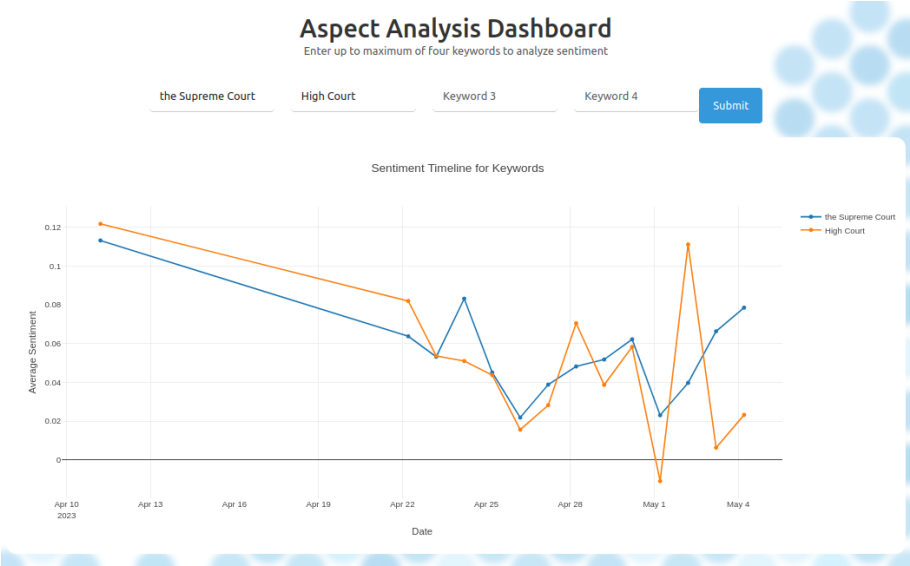
The side panel on the right lists detailed news items, including their focus time, creation date, and associated categories. For example, the panel includes news such as "U.S., Pakistan discuss 'recent events in region'" with a focus on Islamabad on April 2, 2024, created on April 19, 2024, and categorized under topics like Iran, states, United States, Israel, Pakistan, and Blome. Another news item listed is "Pakistan, Iran agree to boost trade, economic cooperation," focused on Islamabad on April 24, 2024, created on April 25, 2024, and categorized under trade, border, economic, Iran, cooperation, and Pakistan. Additionally, there is news about "Raisi, Shehbaz jointly inaugurate Iran Avenue in Islamabad," with a focus on Islamabad on April 22, 2024, created on April 23, 2024.

The sentiment dashboard allows users to input up to four keywords to analyze sentiment trends over time as shown in figure 24. The points are different news articles with the selected keyword present and their sentiment ranging from −1 to 1. This basically shows the sentiment of recommended news based on the aspects selected i.e. keywords. This can also be used to observe the impact of certain keywords on other keywords over a time period. In the Fig. 24e, the sentiment timeline for three keywords—PTI, PML, and PPP—is shown from April 10, 2023, to May 4, 2023.

The sentiment for PTI starts relatively high but shows a steady decline until about April 19, where it then fluctuates with a downward trend. PML's sentiment shows more variability, with peaks and troughs throughout the timeline, ending on a lower



(a) Sentiment of a single keyword



(b) Sentiment of two Keywords, Supreme Court and High Court

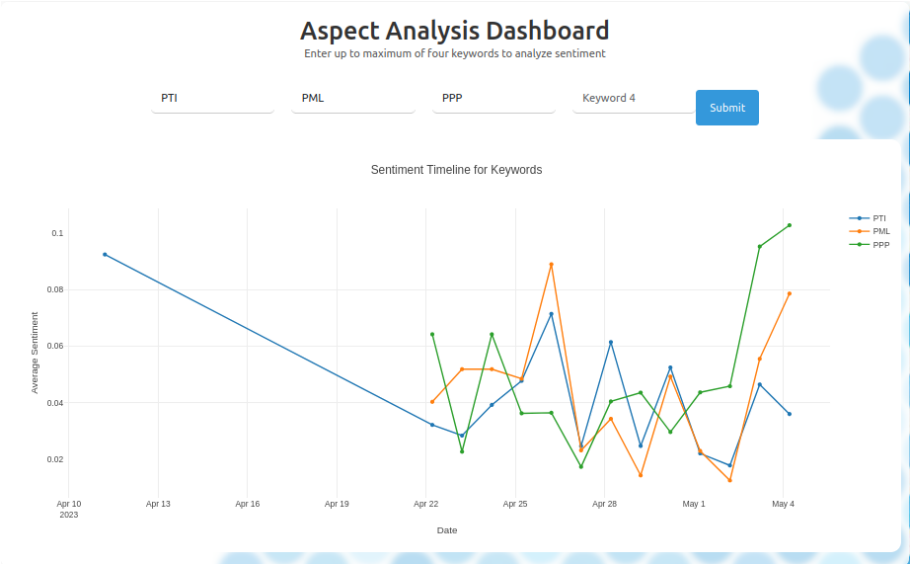
note around April 28 before showing some recovery. PPP's sentiment remains relatively low initially but exhibits a notable upward trend towards the end of the period, particularly around May 1 and May 4.

Overall, the data suggests varying public sentiment towards these political parties over the given period, with PPP showing a significant positive shift towards the end of the timeline, while PTI and PML exhibit more fluctuating and generally declining trends. This sentiment analysis can provide valuable insights into public opinion and the effectiveness of political messaging during this period.

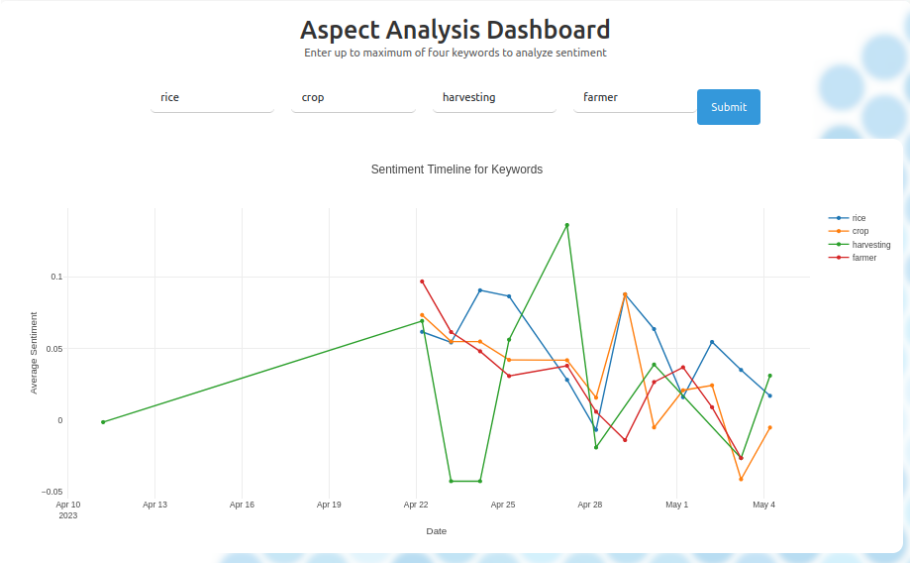
The sentiment timeline tracks four keywords: ”rice,” ”crop,” ”harvesting,” and ”farmer” from April 10, 2023, to May 4, 2023 is shown in Fig. 24d.

The sentiment for ”rice” (blue line) shows moderate fluctuations, peaking towards the end of April and early May. ”Crop” (orange line) also displays variability, with notable peaks and troughs, especially around the end of April. The sentiment for ”harvesting” (green line) starts at zero, then exhibits significant upward and downward movements, with a sharp increase towards the end of April followed by a decline in early May. ”Farmer” (red line) shows a decline initially, followed by fluctuating sentiment with some recovery around April 25, but it dips again towards early May.

Overall, the data illustrates diverse sentiment trends for these agricultural keywords, reflecting



(c) Sentiment of for keywords referring to multiple political parties



(d) Sentiment of for keywords referring to multiple local crops

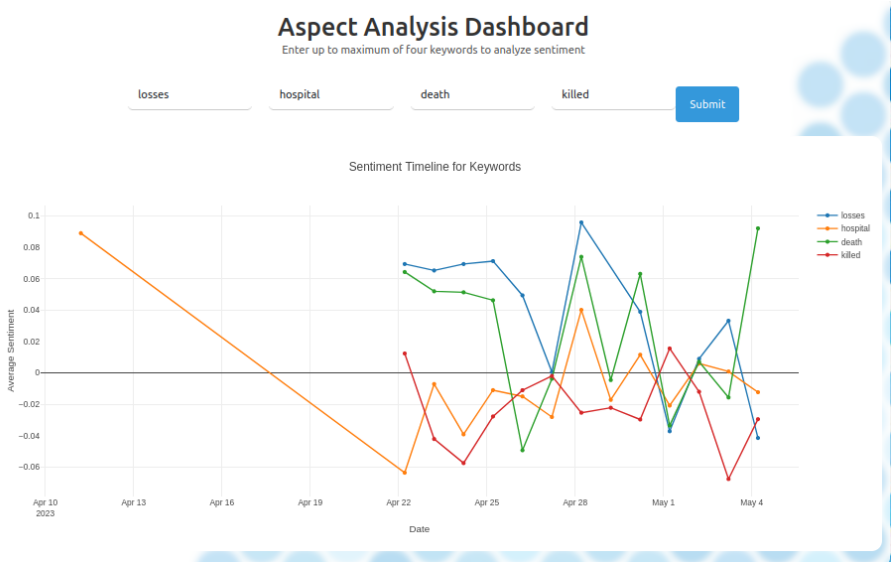
varying public opinions and potential events affecting the agricultural sector during this period. These insights can be critical for stakeholders in agriculture, helping them understand and respond to public sentiment and related issues effectively.

The figure displays a sentiment analysis timeline for the keywords ”losses,” ”hospital,” ”death,” and ”killed” from April 10, 2023, to May 4, 2023. Each keyword’s sentiment is represented by a distinct colored line: blue for ”losses,” orange for ”hospital,” green for ”death,” and red for ”killed.”

Initially, the sentiment for ”hospital” starts positively but shows a sharp decline until mid-April, after which it fluctuates around neutral. The sentiment for ”losses” also starts relatively stable but exhibits high variability, including significant positive spikes at the end of April and early May. The keyword ”death” maintains a mostly negative sentiment throughout the timeline with minor fluctuations, showing some recovery towards the end of April. The sentiment for ”killed” demonstrates high volatility, with both negative and positive spikes, particularly towards the end of the observed period.

A. Entity Relationship Graph: Russia-Ukraine War

Figure 25 presents a comprehensive output generated by the NAaaS platform in response to a query regarding the ongoing Russia-Ukraine war. This visualization showcases the power of integrating Named Entity Recognition (NER) and Retrieval-Augmented Generation (RAG) to provide



(e) Sentiment of for keywords referring to multiple prominent keywords

Fig. 24: Sentiment analysis for different keywords

both relational insights and narrative summaries from a corpus of news articles.

At the top of the figure, multiple tags are extracted from various articles, including `Ukraine anniversary`, `Unwinnable war`, and `Ukrainians and Russians fight for every yard`, all of which contextualize the conflict from political, historical, and combat perspectives. These tags are derived from semantic analysis and serve as high-level thematic descriptors.

The middle section contains a concise RAG-based summary that draws information from multiple trusted news sources. This generated narrative provides a factual overview, mentioning that the conflict began with Russia's invasion of Ukraine in February 2022 and has since reached a military stalemate. The system also includes citation references with hyperlinks (e.g., from Dawn News), showcasing its ability to combine factual content with source traceability.

Most significantly, the lower part of the figure illustrates an **Entity Relationship Graph**. This graph visually maps out key entities mentioned across the news corpus. Central nodes such as `Ukraine`, `Russia`, and `US` are shown in larger circles, denoting their high frequency and centrality in the corpus. Smaller nodes such as `Putin`, `Zelensky`, `IMF`, `Kherson`, `Telegram`, and `Wagner` connect to the main entities via labeled edges that describe their relationships (e.g., *said*, *accused*, *exported*, *signed*, etc.).

This output exemplifies the system's ability to extract structured knowledge from unstructured news text. By combining NLP tasks such as NER, co-reference resolution, and relationship extraction, NAaaS constructs a dynamic knowledge graph that can be used to explore complex geopolitical scenarios and their actors. Moreover, this output demonstrates how NAaaS supports both visual exploration and narrative summarization, empowering researchers, journalists, and policymakers to navigate high-volume news data effectively.

header	dis
Wapda, KRL in thrilling last-ball tie	ABBOTTABA
Presidentâ€™s Cup to mark return of departmental one-day cricket	ABBOTTABA
PM Shehbaz visits family of martyred customs officer	ABBOTTABA
Hospital board takes major decisions despite ban	ABBOTTABA
Man kills wife, her â€˜friendâ€™ for honour	ATTOCK
Nine die, several injured in separate incidents	ATTOCK
Man found guilty of murdering friend	ATTOCK
Teenager stabs friend to death in Hazro	ATTOCK
Two killed, one injured in separate incidents	ATTOCK
â€˜Say no to plasticâ€™ drive launched in Attock	ATTOCK
Golra-Attock Khurd Safari tourist train relaunched	ATTOCK
Two motorcycle lifters arrested	ATTOCK
Nine die, several injured in separate incidents	ATTOCK
Attockâ€™s district monitoring officeâ€™s Facebook page hacked	ATTOCK
Book lovers throng fair in Hassanabdal	ATTOCK
Attockâ€™s district monitoring officeâ€™s Facebook page hacked	ATTOCK
Attock health department vehicles sold in scrap	ATTOCK
Candidates allotted symbols for PA seats in Attock	ATTOCK
Attock health department vehicles sold in scrap	ATTOCK

Fig. 26: Dawn News Bias score

header	district	creation_date	racial_bias	ha
Bahawalnagar plagued by rising crime	BAHAWALNAGAR	08/01/2024 0:00	0.372525483	
ICCI donates Rs20m to Pakistan Sweet Homes	CHAKWAL	10/04/2023 0:00	0.082497932	
Rs40m heist triggers police jurisdiction spat	CHAKWAL	11/01/2024 0:00	0.162101686	
Goods â€˜worth Rs30mâ€™ gutted in mysterious fir	HYDERABAD	10/04/2023 0:00	0.03471512	
PMâ€™s aide slams PTIâ€™s â€˜false propagandaâ	ISLAMABAD	10/04/2023 0:00	0.238933817	
NCHR, HEIs team up to promote rights education	ISLAMABAD	11/01/2024 0:00	0.022846302	
Tribunal clears key PTI candidates	ISLAMABAD	11/01/2024 0:00	0.210431963	
Poultry prices go up by 30pc	ISLAMABAD	11/01/2024 0:00	0.214208394	
Suri-era well still serves villagers	KHUSHAB	10/04/2023 0:00	0.241009429	
Working women still hamstrung	LAHORE	08/01/2024 0:00	0.025671786	
Visitors flock to Bhagat Singhâ€™s village	LAHORE	08/01/2024 0:00	0.167028919	
Man kills daughter to â€˜appeaseâ€™ new wife	LARKANA	10/04/2023 0:00	0.756571352	
Another young man falls prey to ruthless street crimi	LARKANA	10/04/2023 0:00	0.5295192	
Citizens to face Rs1,000 fine over littering	MULTAN	10/04/2023 0:00	0.039565776	
Multanâ€™s citizens embrace harsh winter	MULTAN	08/01/2024 0:00	0.07634446	
Policing failure: Unregistered bikes plying roads	PESHAWAR	10/04/2023 0:00	0.100829676	
Operation begins in â€˜katchaâ€™ area	RAHIM YAR KHAN	10/04/2023 0:00	0.490155786	
Police barred from raids after killing of youth	RAWALPINDI	10/04/2023 0:00	0.064766943	
Pindi announces amnesty for Qingqis	RAWALPINDI	11/01/2024 0:00	0.032408789	

Fig. 27: Tribune News Bias score

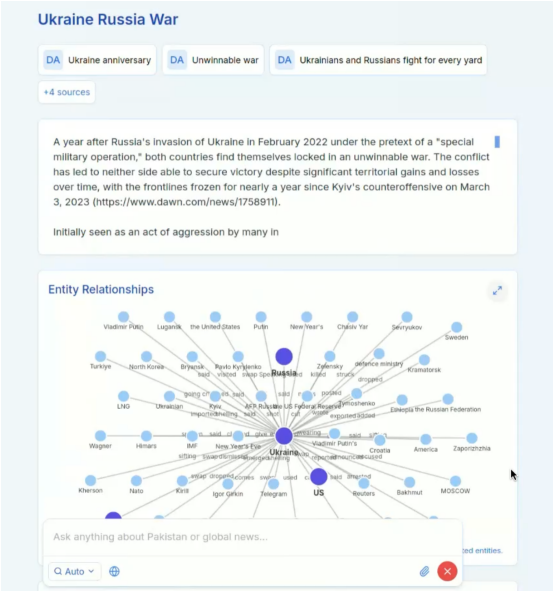


Fig. 25: Entity relationship graph and RAG-based summary generated for the query `Russia Ukraine War` using NAaaS. Central entities and their semantic relationships are extracted from news articles.

B. Media Bias Detection

Using a well-known set of data [51], a computer model was trained on a device using the Roberta model. Roberta uses a technique called Byte-Pair Encoding (BPE) to break words down into smaller parts, called subwords. This helps the computer understand and process text more effectively, especially when it encounters rare or new words it hasn't seen before. The model then uses this processed text to classify the content into one of two categories: biased or neutral.

The model was trained on a variety of data obtained from different news sources, articles, and posts. The data was organized into different CSV files and passed to the model as its training data. Figure 17 illustrates the data pre-processing steps. After pre-processing, the model provides its results in the form of percentages, which are saved in a column in the database as shown in Figure 26 and 27.

Figure **??** presents an analysis of bias within news headlines, focusing specifically on racial, hate speech, and political dimensions. The data originates from two Pakistani newspapers, *Dawn* and *Tribune*.

The figures consists of two tables, each representing one newspaper. Each row within the tables corresponds to a specific news headline, accompanied by:

- **District:** The geographical location associated with the news story.
- **Creation Date:** The date and time the news article was published.
- **Racial Bias:** A score indicating the level of potential racial bias present in the headline.
- **Hate Speech:** A score reflecting the potential presence of hate speech in the headline.
- **Political Bias:** A score evaluating the level of potential political bias within the headline.

Key Points to Note:

- **Purpose:** The figure aims to demonstrate a preliminary assessment of bias within the selected news sources.
- **Score Interpretation:** While the specific meaning of the bias scores isn't provided, it's assumed that higher scores suggest a greater likelihood of the specific bias being present.
- **Limitations:** It's crucial to acknowledge that these are preliminary findings. The accuracy and interpretation of these scores rely heavily on the underlying ML models and their training data.

The results provide an intriguing glimpse into potential biases within news headlines. However, further analysis, model transparency, and a deeper understanding of the scoring methodology are necessary to draw concrete conclusions.

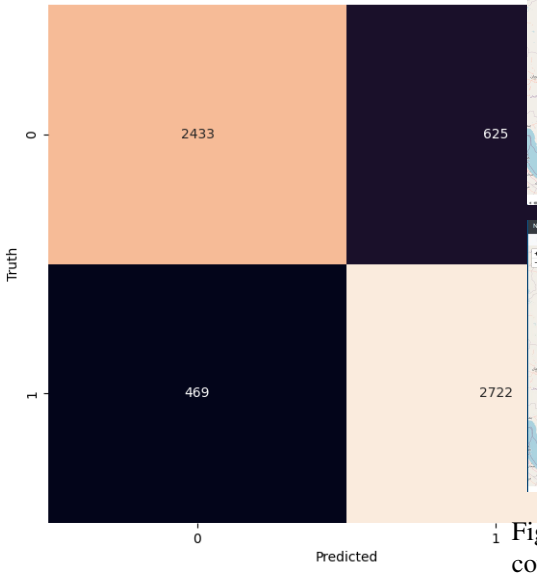


Fig. 28: Confusion Matrix Depicting Media Bias Classification Accuracy

1) Confusion Matrix: False positives occur when the model incorrectly identifies an instance as belonging to the positive class when it actually belongs to the negative class. The model was trained using a diverse mix of articles and statements from various news sources about different cultures, with appropriate labeling. However, it was observed that the majority of the news sources were Western-based, which introduced a distinct perspective bias. This bias significantly contributed to the prominence of false positives. The figure below provides examples of false positives categorized into Media bias using Racial Bias,Hate Speech and Political Bias as a reference instances as shown in Fig. 28.

As shown in Fig. 29 the red bars i the right panel show the bias

- **Top Left:** This snapshot displays the analysis for the period from May 1, 2023, to June 30, 2023. It includes news related to court cases and elections, with bias scores prominently highlighted.
- **Top Right:** This snapshot covers the period from May 1, 2023, to May 31, 2023. It focuses on elections and related comments, showcasing detected biases in news headlines.
- **Bottom Left:** This snapshot represents the analysis for February 1, 2024, to February 29, 2024. It includes news about election outcomes and court hearings, with bias scores indicated.

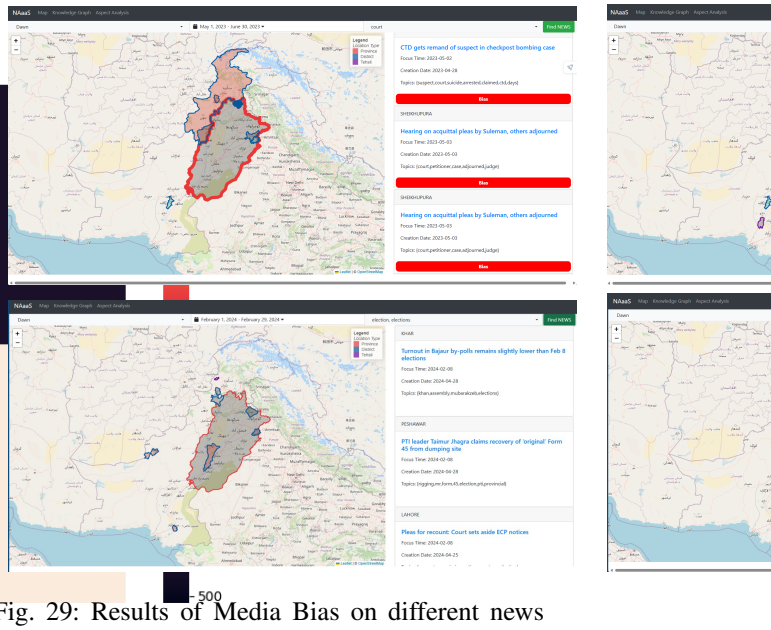


Fig. 29: Results of Media Bias on different news coverage.

- **Bottom Right:** This snapshot illustrates the analysis from January 1, 2024, to April 30, 2024. It highlights news on judicial proceedings and political events, with bias scores displayed.

VI. Deliverables

No.	Description	Status
1	Comprehensive literature reviews the existing techniques, Synthesis of the existing Techniques with reconciliation with state-of-the-art work	Complete Section II
2	Data acquisition process for geo-spatial data for limited area for proof of concept	Complete Section III-A, Table I
3	Detailed system architecture design and formulating of the system model. Feature engineering for temporal and spatial aspect.	Complete Section III
4	Feature engineering for temporal and spatial aspect Multiple temporal and geo tagging techniques will be tried and analyzed for accuracy	In-Progress Section III-C
5	Proof of concept will be developed for extraction of accurate geo-temporal information Detail investigation of deep learning techniques followed by the implementation of ML components.	Complete Section III-H
6	Embedding of developed techniques into web framework	To be Initiated
7	Experimentation with scalable visualization techniques Packaging the solution as one unified on Apache Storm/Spark	In-progress Section IV
8	Paper write-up and submission	In progress
9	Final Report	Completed

VII. Conclusion and Future Work

The architecture of News Analytics as a Service (NAaaS) has been meticulously designed to seamlessly integrate the processes of scraping, parsing, and analyzing news articles from a variety of online sources. By leveraging advanced techniques such as web scraping, Natural Language Processing (NLP), and Kafka-based message queuing, NAaaS provides a comprehensive real-time news analysis solution.

1) **Data Acquisition:** Utilizing web scraping techniques, NAaaS efficiently retrieves structured data from news websites, including headlines, content, publication dates, and geographic locations. This systematic approach ensures a robust and comprehensive dataset for subsequent analysis.
2) **Parsing and NLP:** The parser, built with Python, incorporates text preprocessing, topic modeling, sentiment analysis, and location and time extraction. These functionalities enable the extraction of meaningful insights from unstructured textual data, facilitating advanced analysis such as trend identification and sentiment timelines.
3) **Kafka Integration:** The use of Kafka for message queuing ensures efficient handling of large volumes of data. The Kafka Producer-Consumer architecture enables seamless data transmission and real-time processing, enhancing the system's scalability and reliability.
4) **Knowledge Processing:** The sophisticated knowledge processing engine of NAaaS employs Named Entity Recognition (NER) to identify and classify entities within news articles. This capability is critical for understanding the context and relationships between different entities, supporting tasks such as event detection and information retrieval.
5) **Sentiment and Bias Detection:** By implementing sentiment analysis and media bias detection, NAaaS provides valuable insights into public opinion and potential biases in news reporting. This functionality is crucial for identifying and addressing issues related to misinformation and biased reporting.
6) **Web-GIS Integration:** The final stage of NAaaS involves the visualization of extracted insights using a Web-based Geographic Information System (GIS). This dynamic platform allows users to explore the geographical context of news events interactively, providing a deeper understanding of spatial trends and relationships.

In future work we will be focusing on the following aspects.

- **Trend Analysis and Anomalies** The system analyzes temporal and spatial trends to identify recurring patterns or anomalies. Spikes or drops in the frequency of mentions of specific keywords or locations are investigated

to understand their causes, such as significant events or changes in public sentiment.

- **Cross-referencing Data** Extracted data is cross-referenced with external datasets (e.g., social media, economic indicators) to validate findings and gain additional context. The knowledge graph is used to identify corroborating or conflicting information across different news sources.
- **Addressing Bias using Language Detection Models:** Building on the findings from the papers [52] [53] [54] [55], future work will focus on improving the model by gathering a better dataset that is more representative of regional news. We will also train the model on this dataset and test different techniques to mitigate racial biases in language detection systems. It is crucial to ensure that classifiers are trained on diverse datasets and use context-based classification, which involves reading the entire text and determining bias based on context, rather than relying primarily on keywords associated with bias.
- **User Feedback and Iteration** Continuous feedback from users is gathered to improve the system's functionality and usability. Changes are implemented based on this feedback to better meet users' needs. Models and algorithms used for extraction and analysis are regularly updated to incorporate the latest advancements and ensure ongoing accuracy.

Overall, NAaaS represents a powerful tool for real-time news analysis, offering users a holistic view of the news landscape. By integrating advanced data acquisition, parsing, and analysis techniques with interactive visualizations, NAaaS enables users to gain actionable insights and make informed decisions based on comprehensive news data. The continuous improvement and updating of the system ensure its ongoing relevance and effectiveness in addressing the evolving challenges of news analytics.

REFERENCES

[1] H. Peng, Y. Yang, W. Liu, Y. Chen, B. Zhang, and W. X. Zhao, "Graphrag: Augmenting language models with graph reasoning," *arXiv preprint arXiv:2404.07751*, 2024.

[2] S. Phillips, K. Lee, and S. E. Hassan, "Graphrag: Enhancing retrieval-augmented generation via graph-based retrieval," *arXiv preprint arXiv:2401.04430*, 2024.

[3] T. Ma, X. Li, and M. Chen, "Hybrid-rag: Leveraging structured and unstructured knowledge for open-domain qa," *arXiv preprint arXiv:2310.04588*, 2023.

[4] G. Mialon, M.-T. L. Hoang, H. Pham, Y. Tay, J. Eisenschlos, M. Rabe, N. Houlsby, E. Reif *et al.*, "Retrieval-augmented generation: A survey," *arXiv preprint arXiv:2301.00375*, 2024.

[5] Z. Guo, Y. Li, D. Xu, J. Zeng, Y. Cao, N. Duan, J. Tang *et al.*, "Graphrag++: Enabling graph reasoning in rag with latent entity structures," *arXiv preprint arXiv:2404.07093*, 2024.

[6] A. Keraghel and N. Bouguila, "Graph convolutional networks for news document clustering," in *Proceedings of the 37th International FLAIRS Conference (FLAIRS)*, 2024.

[7] E. Clemedtson, A. Ramesh, A. Raman, and P. Singh, "Graphraft: Graph-based retrieval-augmented fine-tuning for event-centric sentiment-aware summarization," *arXiv preprint arXiv:2401.01574*, 2025.

[8] R. K. Behera, P. K. Bala, N. P. Rana, and Z. Irani, "Responsible natural language processing: A principlist framework for social benefits," *Technological Forecasting and Social Change*, vol. 188, p. 122306, 2023.

[9] D. Byrne, R. Goodhead, M. McMahon, and C. Parle, "Measuring the temporal dimension of text: an application to policymaker speeches," Central Bank of Ireland, Tech. Rep., 2023.

[10] S. Ghosh, S. Sengupta, and P. Mitra, "Spatio-temporal storytelling? leveraging generative models for semantic trajectory analysis," *arXiv preprint arXiv:2306.13905*, 2023.

[11] S. Yang, X. Li, L. Bing, and W. Lam, "Once upon a *time* in *graph*: Relative-time pretraining for complex temporal reasoning," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 11 879–11 895. [Online]. Available: https://aclanthology.org/2023.emnlp-main.728

[12] O. C. Bordeanu, "From data to insights: Unraveling spatio-temporal patterns of cybercrime using nlp and deep learning," Ph.D. dissertation, UCL (University College London), 2024.

[13] G. Wenzel and A. Jatowt, "An overview of temporal commonsense reasoning and acquisition," *arXiv preprint arXiv:2308.00002*, 2023.

[14] Z. Chen, S. Wu, X. Zhang, and Z. Feng, "Tml: A temporal-aware multitask learning framework for time-sensitive question answering," in *Companion Proceedings of the ACM Web Conference 2023*, 2023, pp. 200–203.

[15] M. Ahmed, H. U. Khan, M. A. Khan, U. Tariq, and S. Kadry, "Context-aware answer selection in community question answering exploiting spatial temporal bidirectional long short-term memory," *ACM Transactions on Asian and Low-Resource Language Information Processing*, 2023.

[16] M. A. Spikes, "Learning to think like a journalist: Examining the pedagogy of journalists as teachers of their profession," Ph.D. dissertation, Northwestern University, 2023.

[17] "Apache kafka docs," https://kafka.apache.org/documentation/, accessed: 2024-05-21.

[18] "Apache spark docs," https://spark.apache.org/docs/3.5.1/, accessed: 2024-05-21.

[19] H. Sousa, R. Campos, and A. Jorge, "Tei2go: A multilingual approach for fast temporal expression identification," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023, pp. 5401–5406.

[20] E. Bandara, X. Liang, P. Foytik, S. Shetty, N. Ranasinghe, K. De Zoysa, and W. K. Ng, "Promize - blockchain and self sovereign identity empowered mobile atm platform," in *Intelligent Computing*, K. Arai, Ed. Cham: Springer International Publishing, 2021, pp. 891–911.

[21] G. M. D'silva, A. Khan, Gaurav, and S. Bari, "Real-time processing of iot events with historic data using apache kafka and apache spark with dashing framework," in *2017 2nd IEEE International Conference on Recent Trends in Electronics, Information & Communication Technology (RTEICT)*, 2017, pp. 1804–1809.

[22] X. Meng, J. K. Bradley, B. Yavuz, E. R. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. B. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar, "Mllib: Machine learning in apache spark," *CoRR*, vol. abs/1505.06807, 2015. [Online]. Available: http://arxiv.org/abs/1505.06807

[23] H. Tariq, H. Al-Sahaf, and I. Welch, "Modelling and prediction of resource utilization of hadoop clusters: A machine learning approach," in *Proceedings of the 12th IEEE/ACM international conference on utility and cloud computing*, 2019, pp. 93–100.

[24] M. Ehrmann, M. Romanello, S. Najem-Meyer, A. Doucet, S. Clematide, G. Faggioli, N. Ferro, A. Hanbury, and M. Potthast, "Extended overview of hipe-2022: Named entity recognition and linking in multilingual historical documents," in *CEUR Workshop Proceedings*, no. 3180. CEUR-WS, 2022, pp. 1038–1063.

[25] E. Manjavacas and L. Fonteyn, "Adapting vs. pre-training language models for historical languages," *Journal of Data Mining & Digital Humanities*, no. Digital humanities in languages, 2022.

[26] C. Peng, F. Xia, M. Naseriparsa, and F. Osborne, "Knowledge graphs: Opportunities and challenges," *Artificial Intelligence Review*, vol. 56, no. 11, pp. 13 071–13 102, 2023.

[27] S. Ji, S. Pan, E. Cambria, P. Marttinen, and S. Y. Philip, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE transactions on neural networks and learning systems*, vol. 33, no. 2, pp. 494–514, 2021.

[28] D. Q. Nguyen, T. D. Nguyen, D. Q. Nguyen, and D. Phung, "A novel embedding model for knowledge base completion based on convolutional neural network," *arXiv preprint arXiv:1712.02121*, 2017.

[29] M. Lakshika and H. Caldera, "Knowledge graphs representation for event-related e-news articles," *Machine Learning and Knowledge Extraction*, vol. 3, no. 4, pp. 802–818, 2021.

[30] M. Honnibal and I. Montani, "spacy 2: Natural language understanding with bloom embeddings, convolutional neural networks and incremental parsing. neural machine translation," in *Proceedings of the Association for Computational Linguistics (ACL)*, 2017, pp. 688–697.

[31] ArXiv, "Graphrag: Integrating graph structure into rag," https://arxiv.org/html/2503.19878v1.

[32] CausalWizard, "How to measure a causal relationship (part 1/2)," https://medium.com/@causalwizard/how-to-measure-a-causal-relationship-part-1-2-b93e447d084c.

[33] C. I. Book, "Identification - getting started with causal inference," https://causalinference.gitlab.io/causal-reasoning-book-chapter3/.

[34] CausaLens, "How can ai discover cause and effect?" https://causalai.causalens.com/resources/white-papers/how-can-ai-discover-cause-and-effect/.

[35] B. Ghosh, "Advanced rag with knowledge graphs," https://medium.com/@bijit211987/advanced-rag-with-knowledge-graphs-24262f289b98.

[36] ArXiv, "Wikicausal: Corpus and evaluation framework for causal relationship extraction," https://arxiv.org/html/2402.01454v2.

[37] N. Authors, "Challenges in clinical trial design," *PMC*, 2018, https://pmc.ncbi.nlm.nih.gov/articles/PMC6235714/.

[38] SciBite, "Addressing common challenges with knowledge graphs," https://scibite.com/knowledge-hub/news/common-challenges-with-knowledge-graphs/.

[39] "Beautiful soup 4," https://beautiful-soup-4.readthedocs.io/en/latest/, accessed: 2024-05-06.

[40] T. Groot Kormelink and I. Costera Meijer, "A user perspective on time spent: Temporal experiences of everyday news use," *Journalism Studies*, vol. 21, no. 2, pp. 271–286, 2020.

[41] M. Si, L. Cui, W. Guo, Q. Li, L. Liu, X. Lu, and X. Lu, "A comparative analysis for spatio-temporal spreading patterns of emergency news," *Scientific Reports*, vol. 10, no. 1, p. 19472, 2020.

[42] H. Jiang, G. Wang, W. Chen, C. Zhang, and B. F. Karlsson, "Boningknife: Joint entity mention detection and typing for nested ner via prior boundary knowledge," *arXiv preprint arXiv:2107.09429*, 2021.

[43] V. Mikhailov and T. Shavrina, "Domain-transferable method for named entity recognition task," *arXiv preprint arXiv:2011.12170*, 2020.

[44] J. D'Souza and S. Auer, "Computer science named entity recognition in the open research knowledge graph," in *International Conference on Asian Digital Libraries*. Springer, 2022, pp. 35–45.

[45] K. Pant, T. Dadu, and R. Mamidi, "Towards detection of subjective bias using contextualized word embeddings," in *Companion Proceedings of the Web Conference 2020*, ser. WWW '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 75–76. [Online]. Available: https://doi.org/10.1145/3366424.3382704

[46] K. Chaloner and A. Maldonado, "Measuring gender bias in word embeddings across domains and discovering new gender bias word categories," in *Proceedings of the First Workshop on Gender Bias in Natural Language Processing*, M. R. Costa-jussà, C. Hardmeier, W. Radford, and K. Webster, Eds. Florence, Italy: Association for Computational Linguistics, Aug. 2019, pp. 25–32. [Online]. Available: https://aclanthology.org/W19-3804

[47] W. Guo and A. Caliskan, "Detecting emergent intersectional biases: Contextualized word embeddings contain a distribution of human-like biases," in *Proceedings of the 2021 AAAI/ACM Conference on AI, Ethics, and Society*, ser. AIES '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 122–133. [Online]. Available: https://doi.org/10.1145/3461702.3462536

[48] K. K. Singh, D. Mahajan, K. Grauman, Y. J. Lee, M. Feiszli, and D. Ghadiyaram, "Don't judge an object by its context: Learning to overcome contextual bias," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[49] S. Kuang and B. D. Davison, "Semantic and context-aware linguistic model for bias detection," in *Proc. of the Natural Language Processing meets Journalism IJCAI-16 Workshop*, 2016, pp. 57–62.

[50] F.-J. Rodrigo-Ginés, J. Carrillo-de Albornoz, and L. Plaza, "A systematic review on media bias detection: What is media bias, how it is expressed, and how to detect it," *Expert Systems with Applications*, p. 121641, 2023.

[51] M. Wessel, T. Horych, T. Ruas, A. Aizawa, B. Gipp, and T. Spinde, "Introducing mbib - the first media bias identification benchmark task and dataset collection," in *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '23)*. New York, NY, USA: ACM, 2023. [Online]. Available: https://media-bias-research.org/wp-content/uploads/2023/04/Wessel2023Preprint.pdf

[52] T. Davidson, D. Bhattacharya, and I. Weber, "Racial bias in hate speech and abusive language detection datasets," *arXiv preprint arXiv:1905.12516*, 2019.

[53] R. Baly, G. Da San Martino, J. Glass, and P. Nakov, "We can detect your bias: Predicting the political ideology of news articles," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, B. Webber, T. Cohn, Y. He, and Y. Liu, Eds. Online: Association for Computational Linguistics, Nov. 2020, pp. 4982–4991. [Online]. Available: https://aclanthology.org/2020.emnlp-main.404

[54] M. P. Cameron, "Two models for illustrating the economics of media bias in a policy-oriented course," *The Journal of Economic Education*, vol. 54, no. 3, pp. 281–288, 2023.

[55] F.-J. Rodrigo-Ginés, "Automated media bias detection: Challenges and opportunities," 2021.